



西安电子科技大学  
XIDIAN UNIVERSITY

# Modern Machine Learning

## Machine Learning (Bilingual)

Yang Li ([liy@xidian.edu.cn](mailto:liy@xidian.edu.cn))

School of Artificial Intelligence  
Xidian University

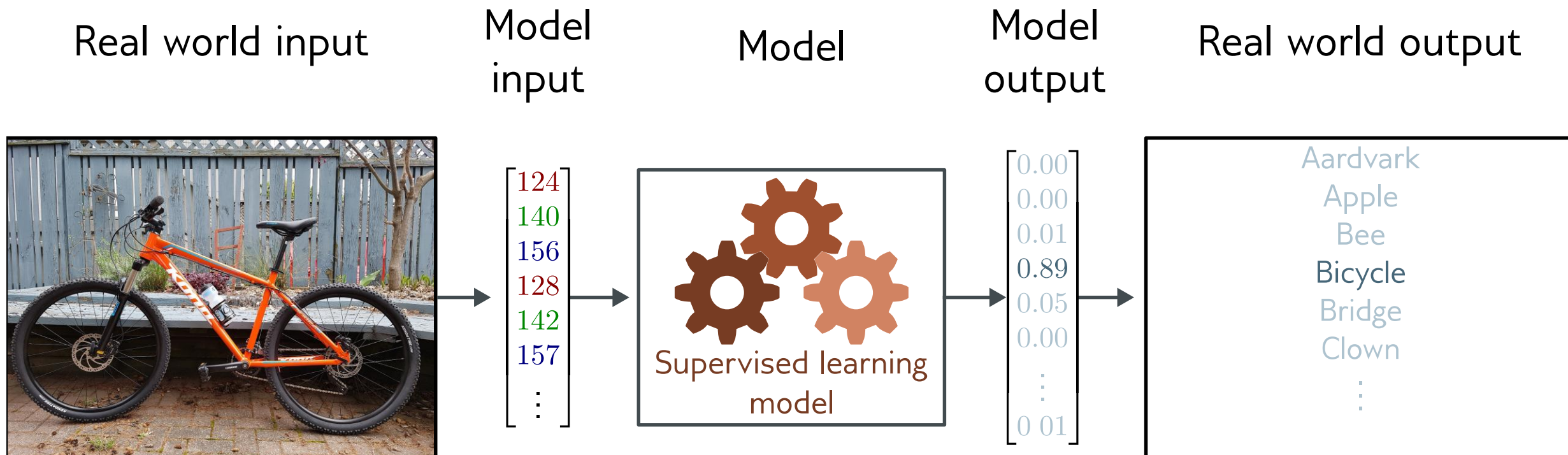
May, 2025



# Convolutional networks #1

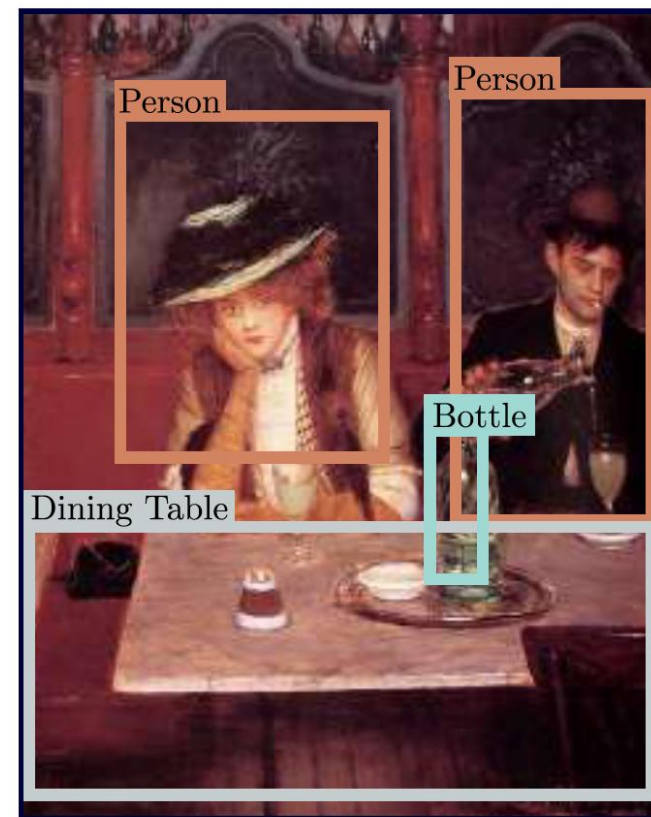
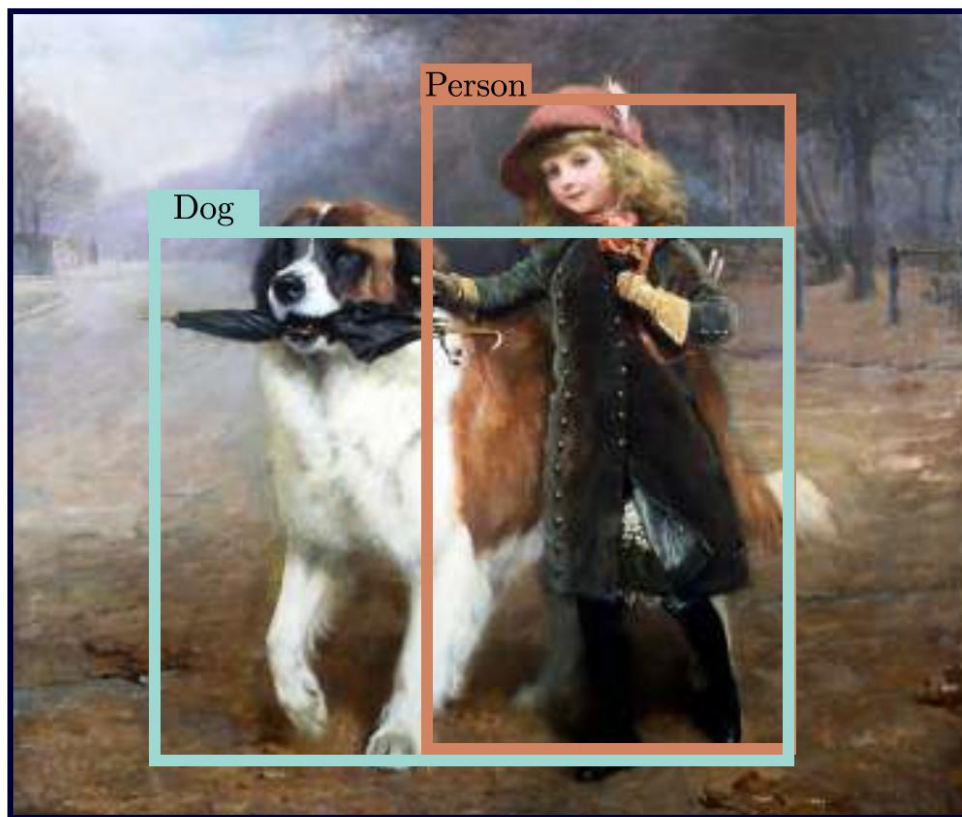
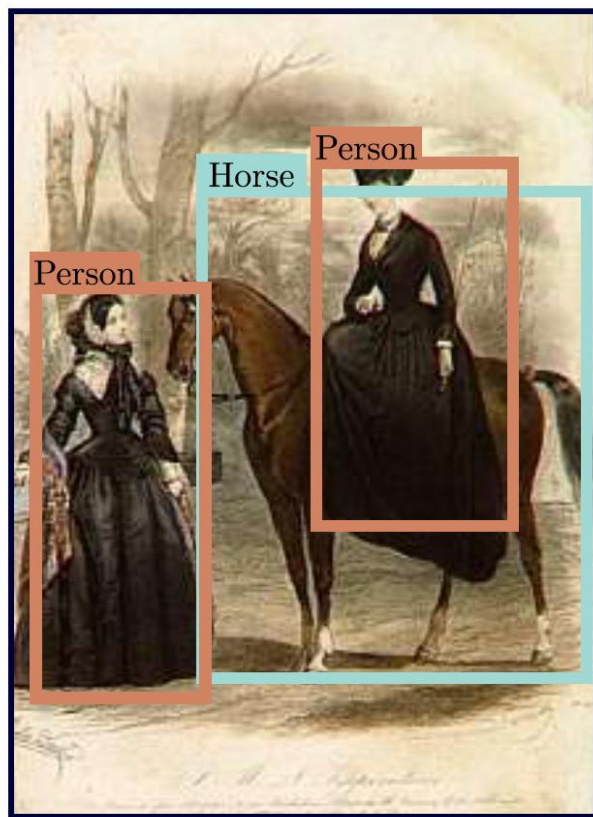
- Networks for images
- Invariance and equivariance
- 1D convolution
- Convolutional layers
- Channels
- Receptive fields
- Convolutional network for MNIST 1D

# Image classification



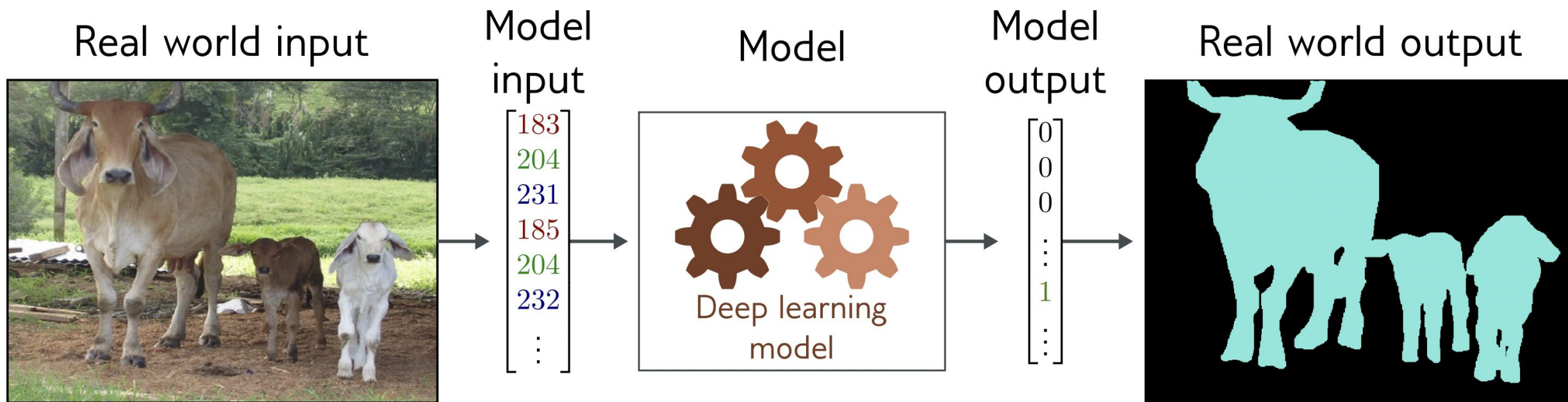
- Multiclass classification problem (discrete classes, >2 possible classes)
- Convolutional network

## Object detection





# Image segmentation



- Multivariate binary classification problem (many outputs, two discrete classes)
- Convolutional encoder-decoder network

# Networks for images

- Images have three properties that suggest the need for specialized model architecture.
- **First**, they are high-dimensional. A typical image for a classification task contains  $224 \times 224$  RGB values (i.e., 150,528 input dimensions).
  - Hidden layers in fully connected networks are generally larger than the input size, so even for a shallow network, the number of weights would exceed  $150,528^2$ , or **22 billion**. This poses obvious practical problems in terms of the required **training data, memory, and computation**.

# Networks for images

- **Second**, nearby image pixels are statistically related.
  - However, fully connected networks have no notion of “nearby” and treat the relationship between every input equally.
  - If the pixels of the training and test images were randomly permuted in the same way, the network could still be trained with no practical difference.
- **Third**, the interpretation of an image is stable under geometric transformations.
  - An image of a tree is still an image of a tree if we shift it leftwards by a few pixels.
  - However, this shift changes every input to the network. Hence, a fully connected model must learn the patterns of pixels that signify a tree separately at every position, which is clearly inefficient.

# Convolutional networks

- Convolutional layers process each **local image region** independently, using **parameters shared** across the whole image.
- They use **fewer parameters** than fully connected layers, exploit the **spatial relationships between nearby pixels**, and don't have to re-learn the interpretation of the pixels at every position.
- A network predominantly consisting of convolutional layers is known as a convolutional neural network or CNN.





# Convolutional networks

- Networks for images
- Invariance and equivariance
- 1D convolution
- Convolutional layers
- Channels
- Receptive fields
- Convolutional network for MNIST 1D

- We argued above that some properties of images (e.g., tree texture) are stable under transformations. In this section, we make this idea more mathematically precise.

## Invariance

- A function  $f[x]$  is **invariant** to a transformation  $t[x]$  if:

$$f[t[x]] = f[x]$$

i.e., the function output is the same even after the transformation is applied.

# Invariance example

e.g., Image classification

- Image has been translated, but we want our classifier to give the same result



- The network  $f[x]$  should **identify** an image as containing the **same object**, even if it has been translated, rotated, flipped, or warped.

# Equivariance

- A function  $f[x]$  is **equivariant** to a transformation  $t[ ]$  if:

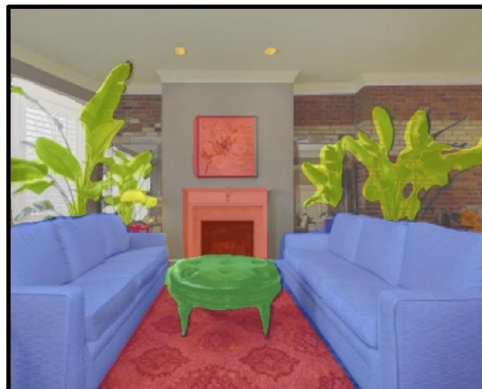
$$\mathbf{f}[\mathbf{t}[\mathbf{x}]] = \mathbf{t}[\mathbf{f}[\mathbf{x}]]$$

i.e., the output is transformed in the same way as the input

## Equivariance example

e.g., Image segmentation

- Image has been translated and we want segmentation to translate with it







## Convolutional networks

- Networks for images
- Invariance and equivariance
- **1D convolution**
- Convolutional layers
- Channels
- Receptive fields
- Convolutional network for MNIST 1D

# Convolution\* in 1D

- Input vector  $\mathbf{x}$ :

$$\mathbf{x} = [x_1, x_2, \dots, x_I]$$

- Output is weighted sum of neighbors:

$$z_i = \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}$$

- Convolutional **kernel** or **filter**:

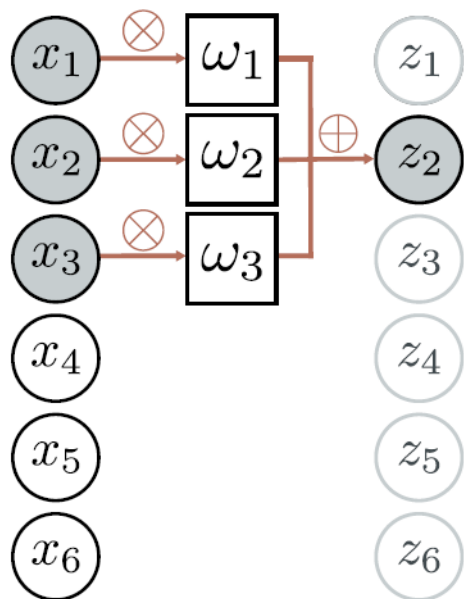
$$\boldsymbol{\omega} = [\omega_1, \omega_2, \omega_3]^T$$


Kernel size = 3

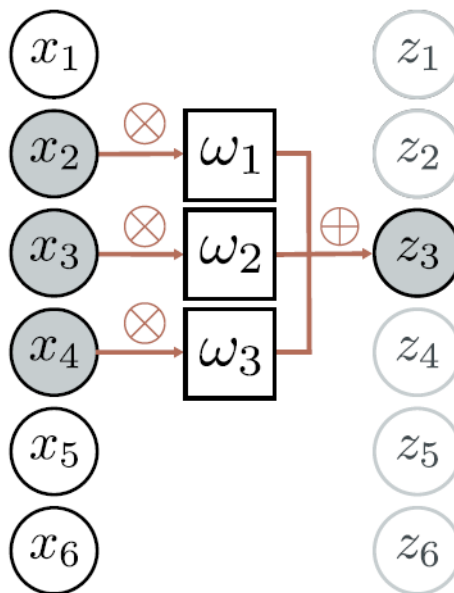
## Convolution with kernel size 3

- If we translate the input  $x$ , then the corresponding output  $z$  is translated in the same way.

a)



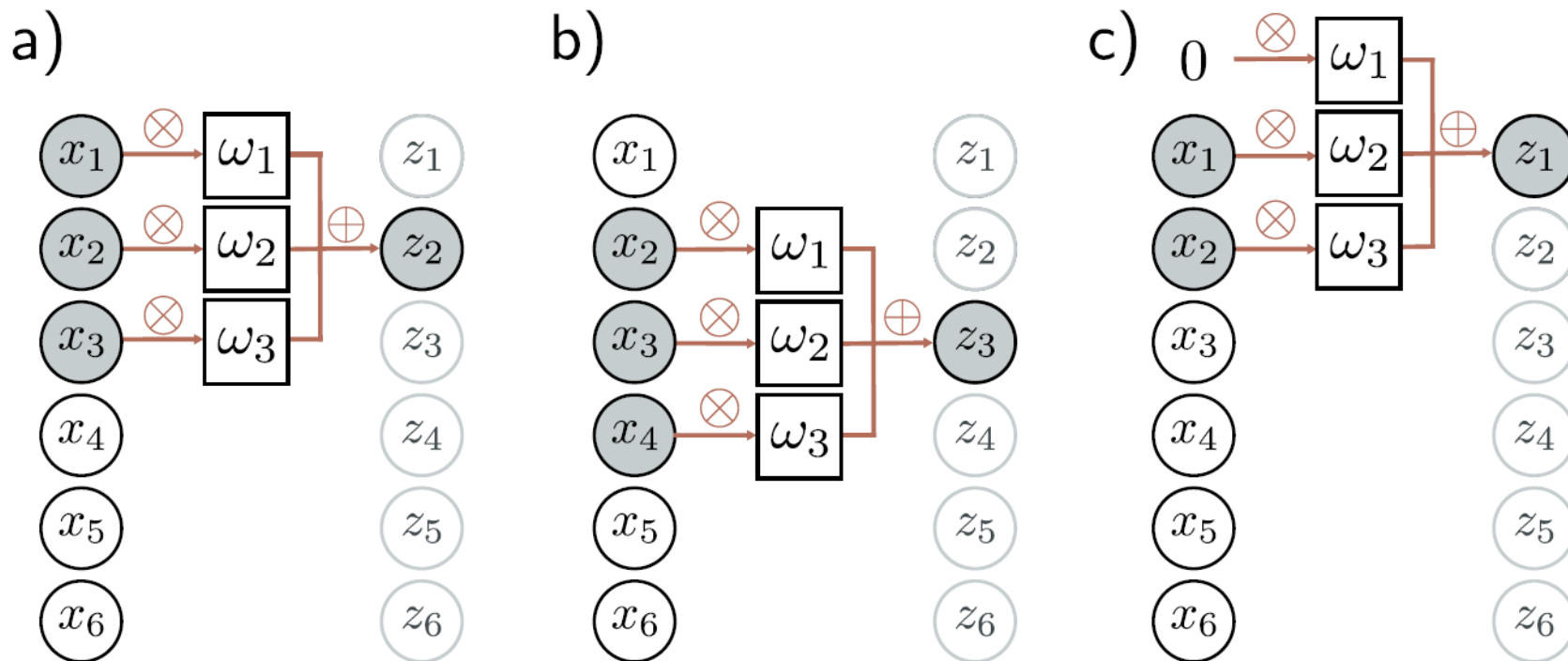
b)



Equivariant to translation of input

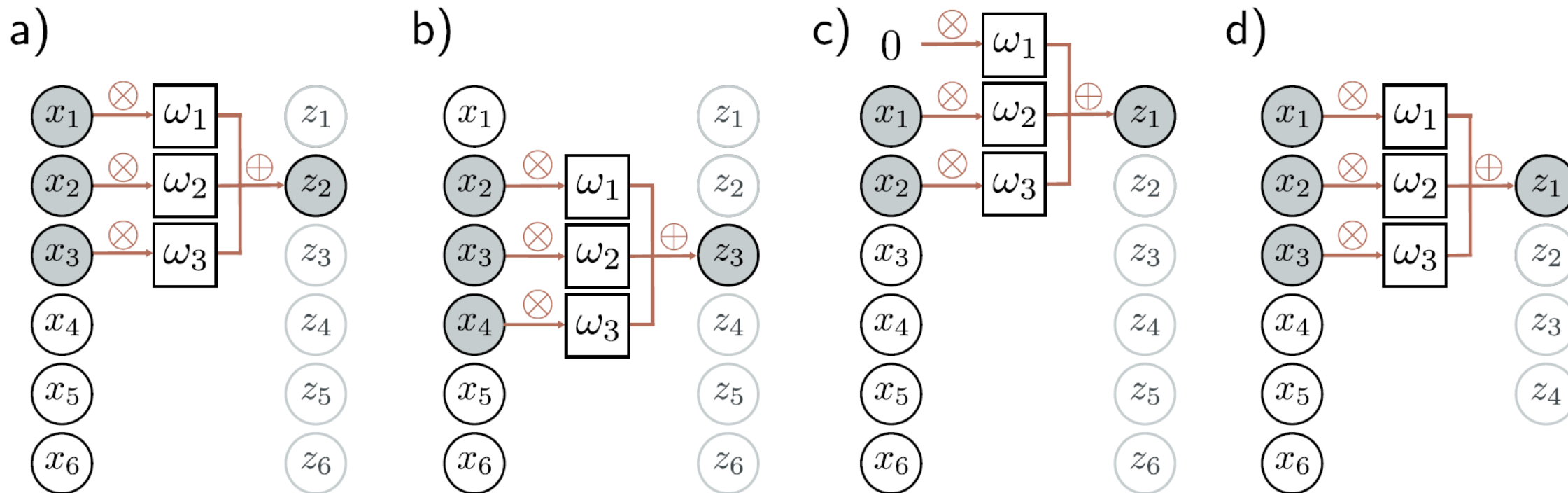
$$\mathbf{f}[\mathbf{t}[\mathbf{x}]] = \mathbf{t}[\mathbf{f}[\mathbf{x}]]$$

## Zero padding



Treat positions that are beyond end of the input as **zero**.

## “Valid” convolutions



Only process positions where kernel falls in image (smaller output).

# Stride, kernel size, and dilation

- **Stride** = shift by  $k$  positions for each output
  - Decreases size of output relative to input
- **Kernel size** = weight a different number of inputs for each output
  - Combine information from a larger area
  - But Increasing the kernel size has the disadvantage of requiring more weights
- **Dilated** or **atrous** convolutions = intersperse kernel values with zeros
  - Combine information from a larger area
  - Fewer parameters



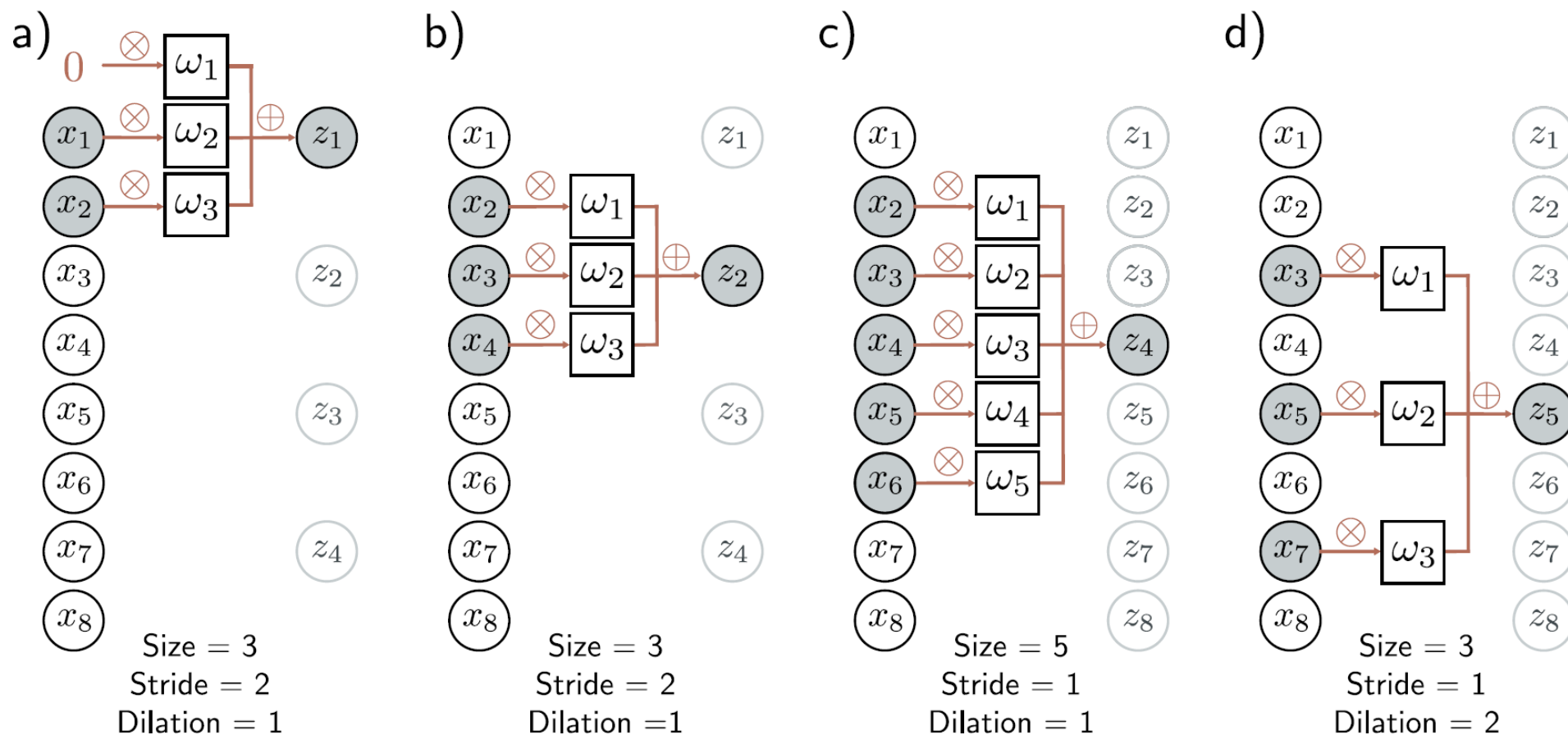


Figure. Stride, kernel size, and dilation. a) With a stride of two, we evaluate the kernel at every other position, so the first output  $z_1$  is computed from a weighted sum centered at  $x_1$ , and b) the second output  $z_2$  is computed from a weighted sum centered at  $x_3$  and so on. c) The kernel size can also be changed. With a kernel size of five, we take a weighted sum of the nearest five inputs. d) In dilated or atrous convolution, we intersperse zeros in the weight vector to allow us to combine information over a large area using fewer weights.



# Convolutional networks

- Networks for images
- Invariance and equivariance
- 1D convolution
- Convolutional layers
- Channels
- Receptive fields
- Convolutional network for MNIST 1D

# Convolutional layer

- A convolutional layer computes its output by convolving the input, adding a bias  $\beta$ , and passing each result through an activation function  $a[\cdot]$ .
- With kernel size three, stride one, and dilation rate one, the  $i$ -th hidden unit  $h_i$  would be computed as:

$$\begin{aligned} h_i &= a[\beta + \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}] \\ &= a\left[\beta + \sum_{j=1}^3 \omega_j x_{i+j-2}\right] \end{aligned}$$

# Special case of fully-connected network

## ■ Convolutional network:

$$\begin{aligned} h_i &= a [\beta + \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}] \\ &= a \left[ \beta + \sum_{j=1}^3 \omega_j x_{i+j-2} \right] \end{aligned}$$

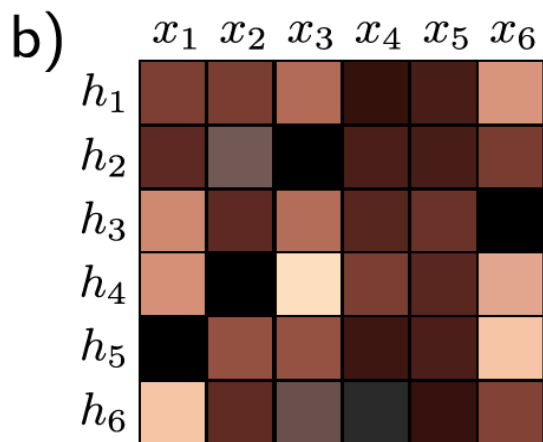
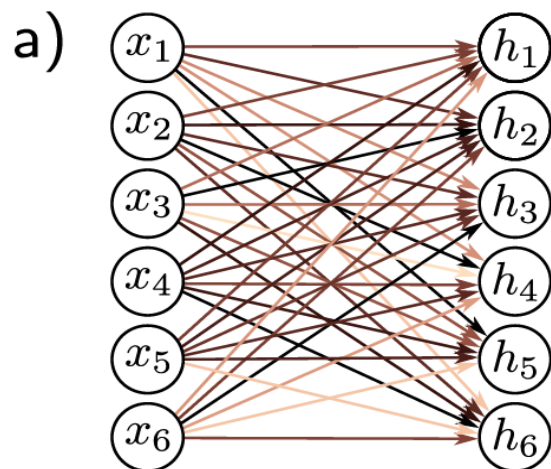
← 3 weights, 1 bias

## ■ Fully connected network:

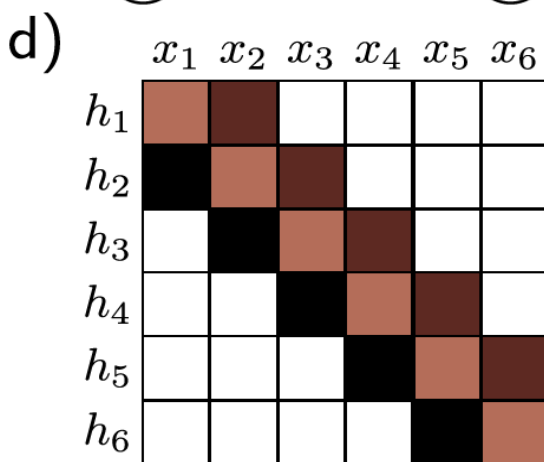
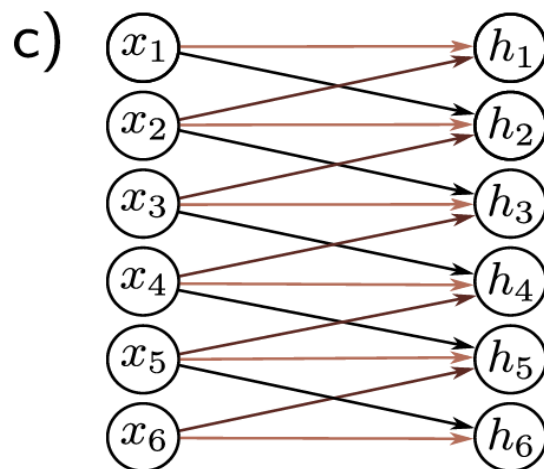
$$h_i = a \left[ \beta_i + \sum_{j=1}^D \omega_{ij} x_j \right]$$

←  $D^2$  weights, D biases

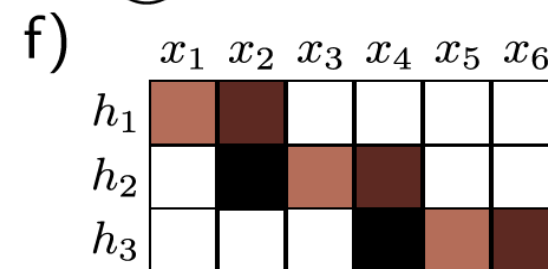
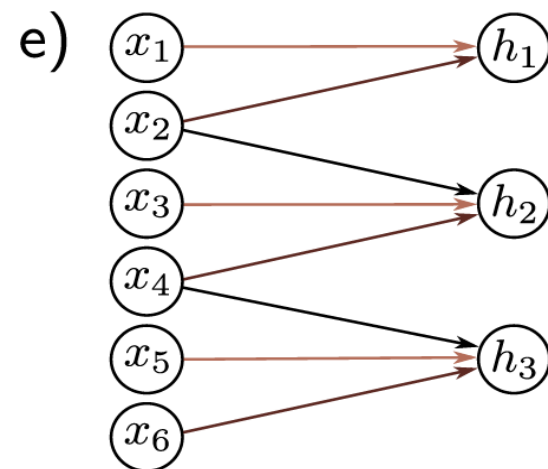
## Special case of fully-connected network



Fully connected network



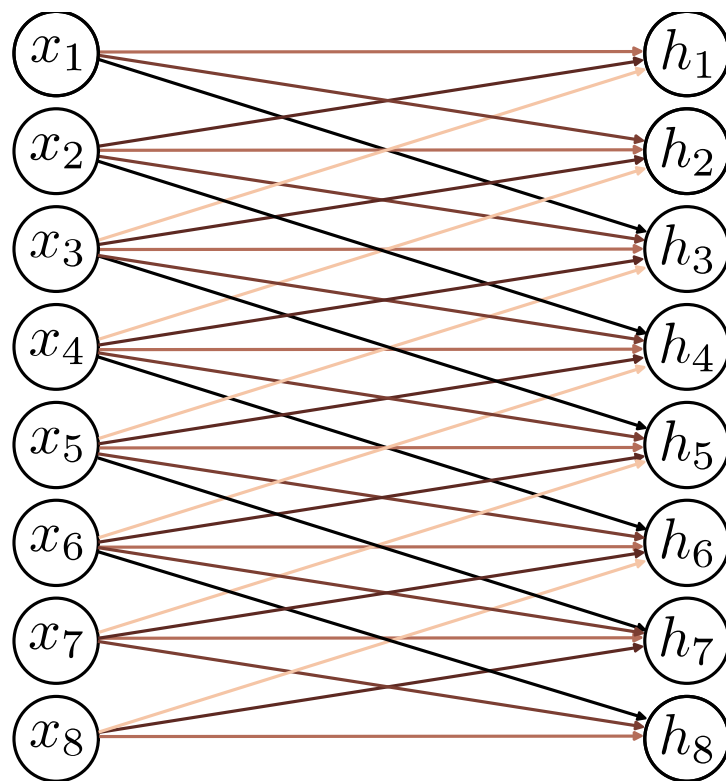
Convolution, size 3, stride 1,  
dilation 1, zero padding



Convolution, size 3, stride 2,  
dilation 1, zero padding

## Question 1

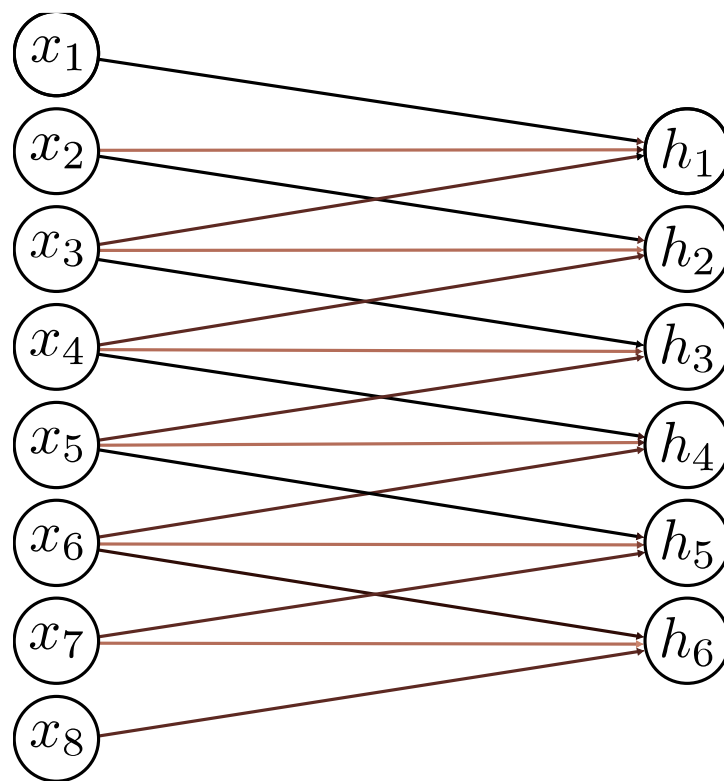
- Kernel size?
- Stride?
- Dilation?
- Zero padding / valid?



|       | $x_1$      | $x_2$      | $x_3$       | $x_4$       | $x_5$       | $x_6$       | $x_7$       | $x_8$       |
|-------|------------|------------|-------------|-------------|-------------|-------------|-------------|-------------|
| $h_1$ | dark brown | dark brown | light brown | white       | white       | white       | white       | white       |
| $h_2$ | dark brown | dark brown | dark brown  | light brown | white       | white       | white       | white       |
| $h_3$ | black      | dark brown | dark brown  | dark brown  | light brown | white       | white       | white       |
| $h_4$ | white      | black      | dark brown  | dark brown  | dark brown  | light brown | white       | white       |
| $h_5$ | white      | white      | black       | dark brown  | dark brown  | dark brown  | light brown | white       |
| $h_6$ | white      | white      | white       | black       | dark brown  | dark brown  | dark brown  | light brown |
| $h_7$ | white      | white      | white       | white       | black       | dark brown  | dark brown  | dark brown  |
| $h_8$ | white      | white      | white       | white       | white       | black       | dark brown  | dark brown  |

## Question 2

- Kernel size?
- Stride?
- Dilation?
- Zero padding / valid?

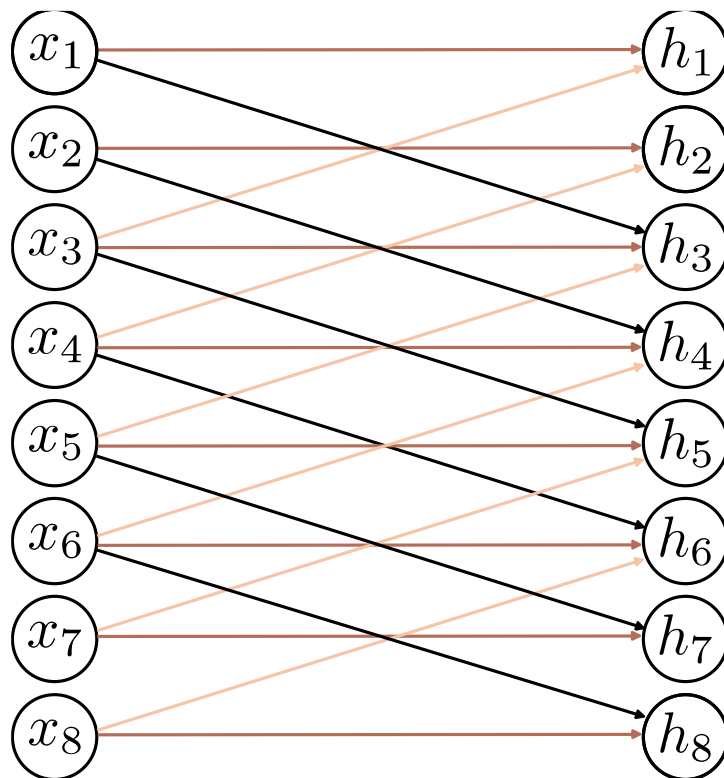


|       | $x_1$ | $x_2$ | $x_3$ | $x_4$ | $x_5$ | $x_6$ | $x_7$ | $x_8$ |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| $h_1$ |       |       |       |       |       |       |       |       |
| $h_2$ |       |       |       |       |       |       |       |       |
| $h_3$ |       |       |       |       |       |       |       |       |
| $h_4$ |       |       |       |       |       |       |       |       |
| $h_5$ |       |       |       |       |       |       |       |       |
| $h_6$ |       |       |       |       |       |       |       |       |



## Question 3

- Kernel size?
- Stride?
- Dilation?
- Zero padding / valid?



|       | $x_1$      | $x_2$      | $x_3$       | $x_4$       | $x_5$       | $x_6$       | $x_7$       | $x_8$       |
|-------|------------|------------|-------------|-------------|-------------|-------------|-------------|-------------|
| $h_1$ | Dark Brown | White      | Light Brown | White       | White       | White       | White       | White       |
| $h_2$ | White      | Dark Brown | White       | Light Brown | White       | White       | White       | White       |
| $h_3$ | Black      | White      | Dark Brown  | White       | Light Brown | White       | White       | White       |
| $h_4$ | White      | Black      | White       | Dark Brown  | White       | Light Brown | White       | White       |
| $h_5$ | White      | White      | Black       | White       | Dark Brown  | White       | Light Brown | White       |
| $h_6$ | White      | White      | White       | Black       | White       | Dark Brown  | White       | Light Brown |
| $h_7$ | White      | White      | White       | White       | Black       | White       | Dark Brown  | White       |
| $h_8$ | White      | White      | White       | White       | White       | Black       | White       | Dark Brown  |



# Convolutional networks

- Networks for images
- Invariance and equivariance
- 1D convolution
- Convolutional layers
- Channels
- Receptive fields
- Convolutional network for MNIST 1D

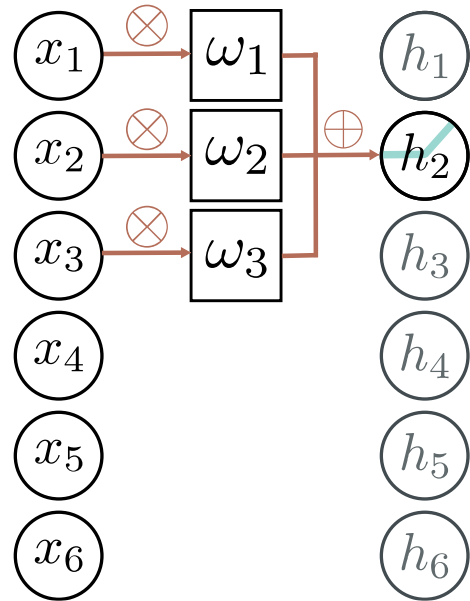
# Channels

- If we only apply a single convolution, information will inevitably be lost;
- we are averaging nearby inputs, and the ReLU activation function clips results that are less than zero.
- Hence, it is usual to compute several convolutions **in parallel**. Each convolution produces a new set of hidden variables, termed a **feature map** or **channel**.

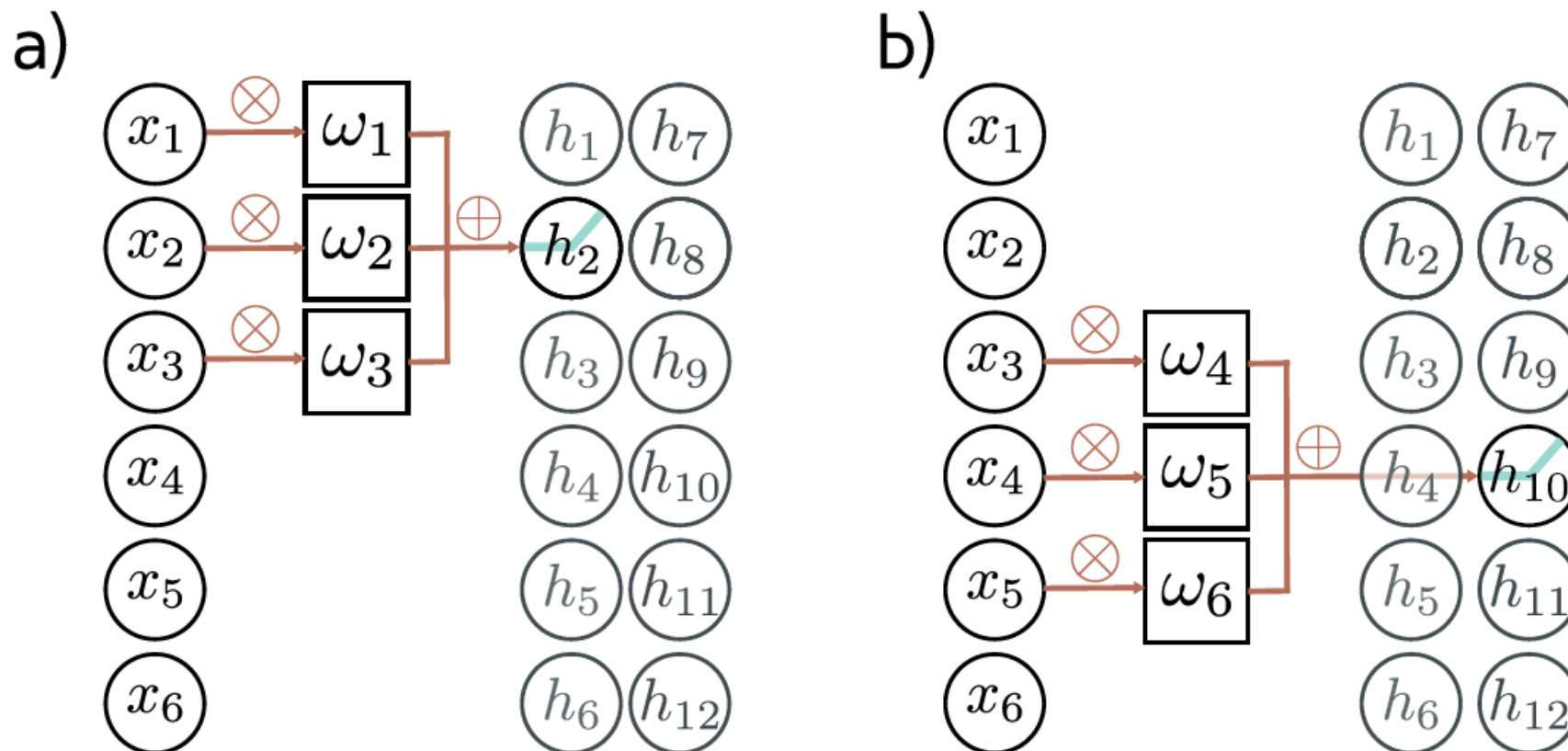


## One output channel, one input channel

a)

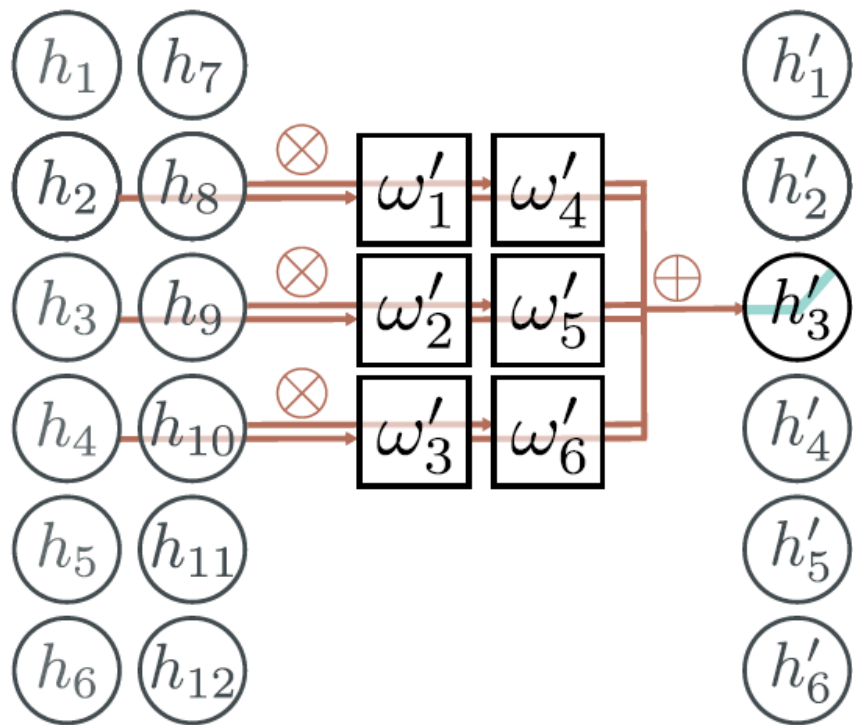


One input channel, two output channels,



# Two input channels, one output channel

c)



# How many parameters?

- If there are  $C_i$  input channels and kernel size  $K$

$$\Omega \in \mathbb{R}^{C_i \times K} \qquad \beta \in \mathbb{R}$$

- If there are  $C_i$  input channels and  $C_o$  output channels

$$\Omega \in \mathbb{R}^{C_i \times C_o \times K} \qquad \beta \in \mathbb{R}^{C_o}$$

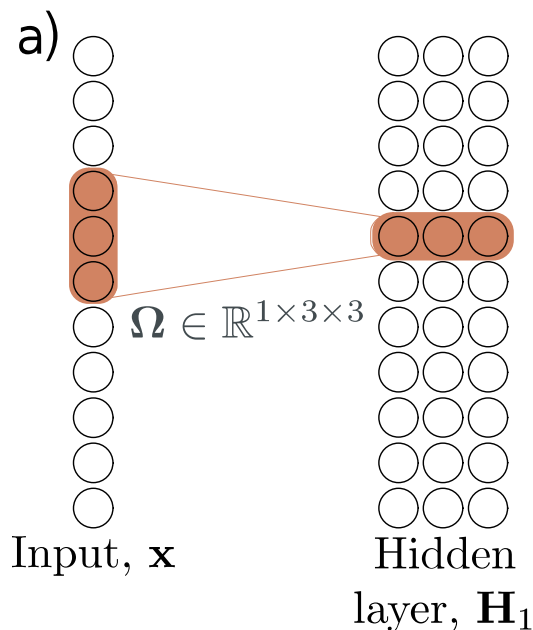


# Convolutional networks

- Networks for images
- Invariance and equivariance
- 1D convolution
- Convolutional layers
- Channels
- Receptive fields
- Convolutional network for MNIST 1D

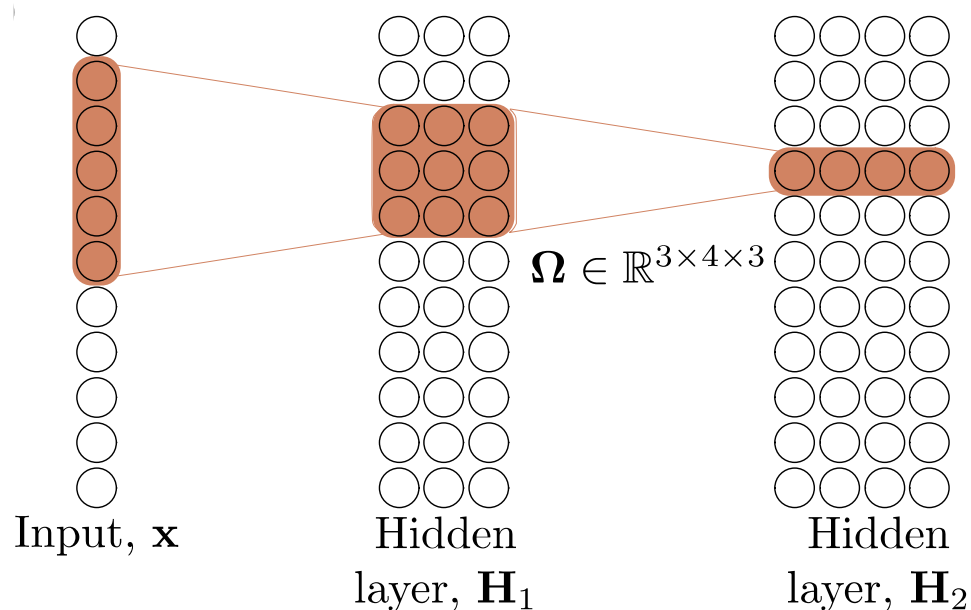
# Receptive fields

- The **receptive field** of a hidden unit in the network is the region of the original input that feeds into it.
- Consider a convolutional network where each convolutional layer has kernel size three. The hidden units in the first layer take a weighted sum of the three closest inputs, so have **receptive fields of size three**.

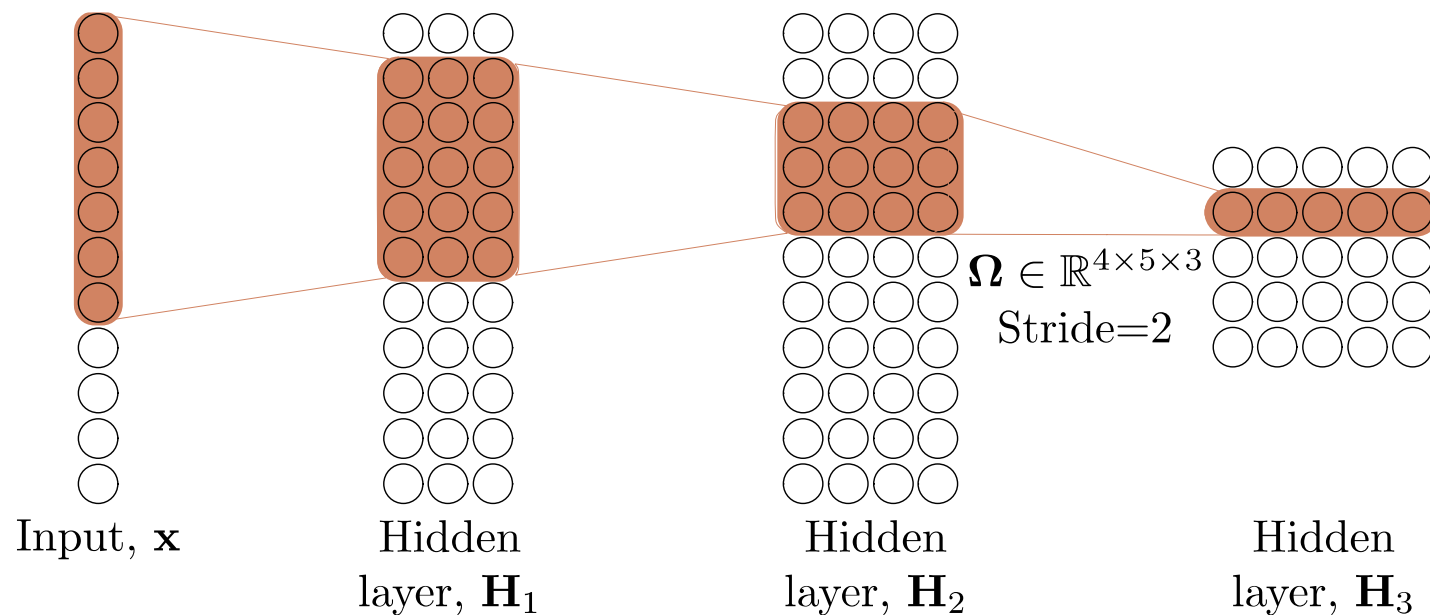


## Receptive fields

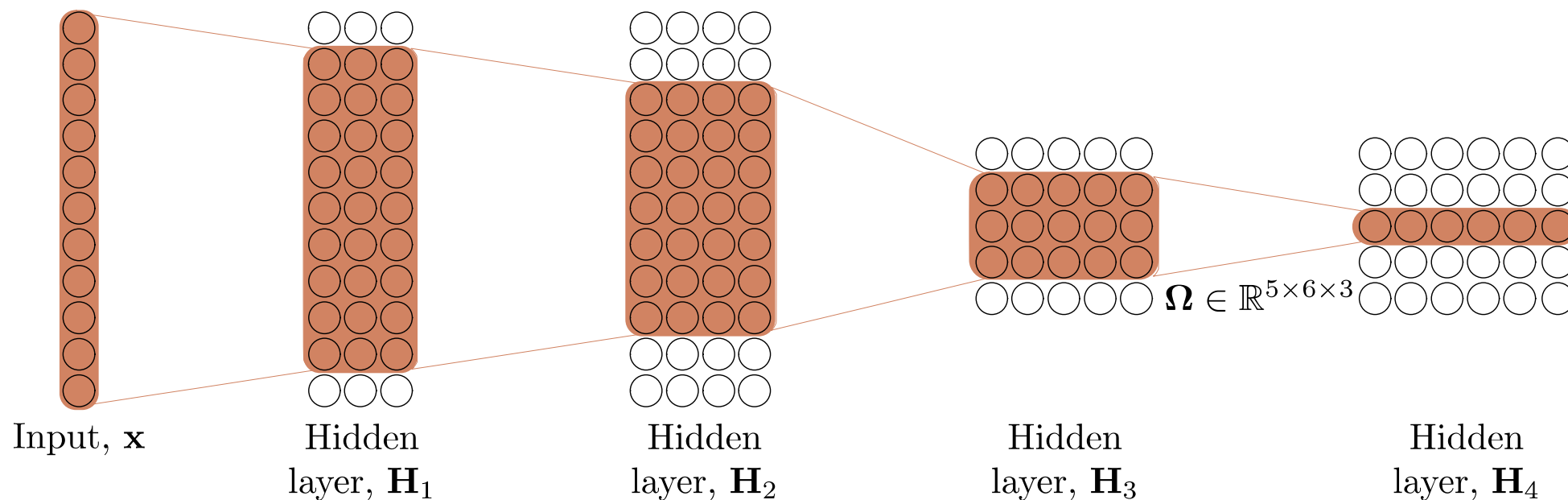
- The units in the second layer take a weighted sum of the three closest positions in the first layer, which are themselves weighted sums of three inputs. Hence, the hidden units in the second layer have **a receptive field of size five**.



# Receptive fields



## Receptive fields





# Convolutional networks

- Networks for images
- Invariance and equivariance
- 1D convolution
- Convolutional layers
- Channels
- Receptive fields
- Convolutional network for MNIST 1D

## MNIST 1D Dataset

Original MNIST examples

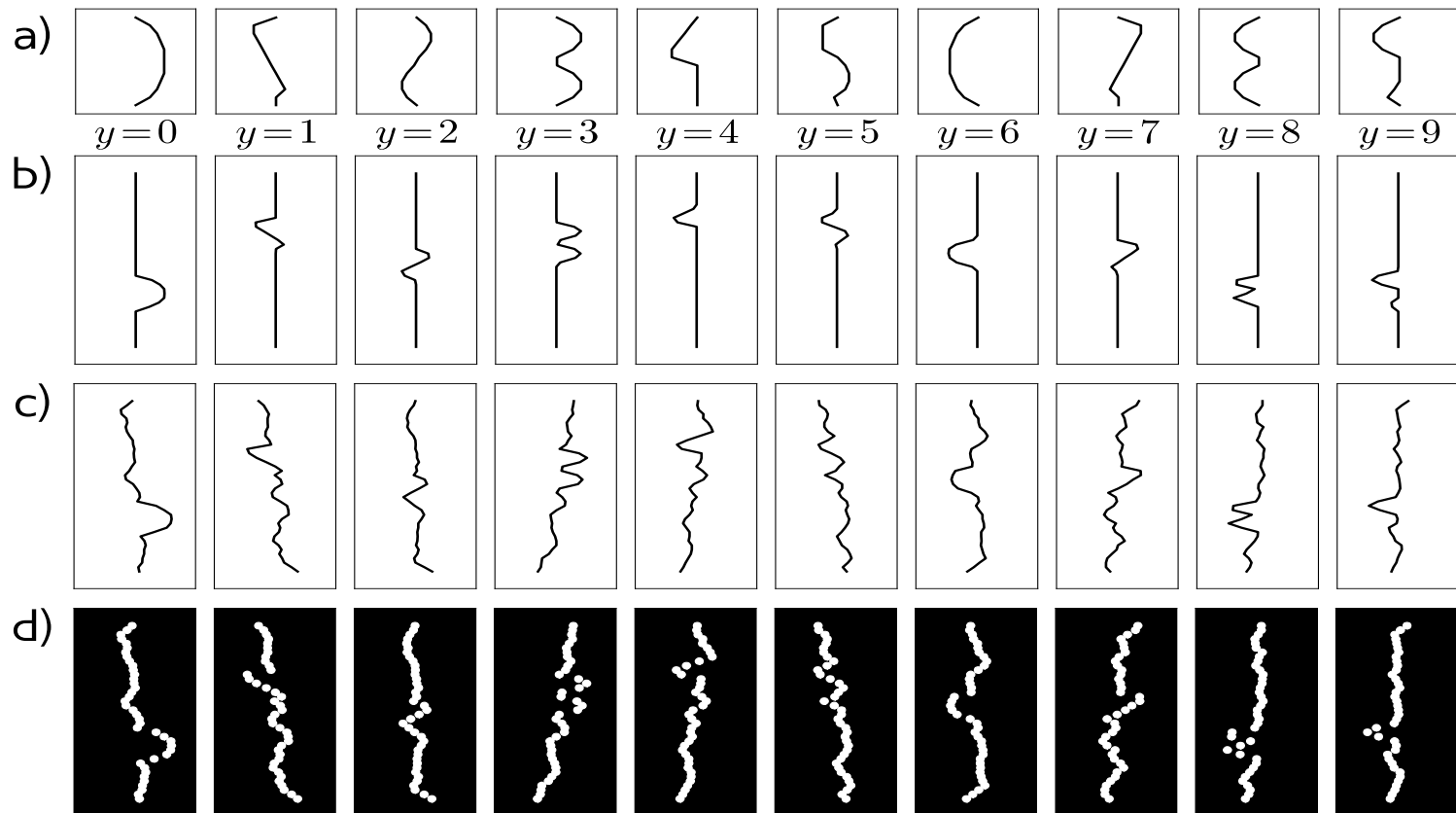


Figure MNIST-1D.

- a) Templates for 10 classes  $y \in \{0, \dots, 9\}$ , based on digits 0–9.
  - b) Training examples  $x$  are created by randomly transforming a template and
  - c) adding noise.
  - d) The horizontal offset of the transformed template is then sampled at 40 vertical positions.
- Adapted from (Greydanus, 2020)

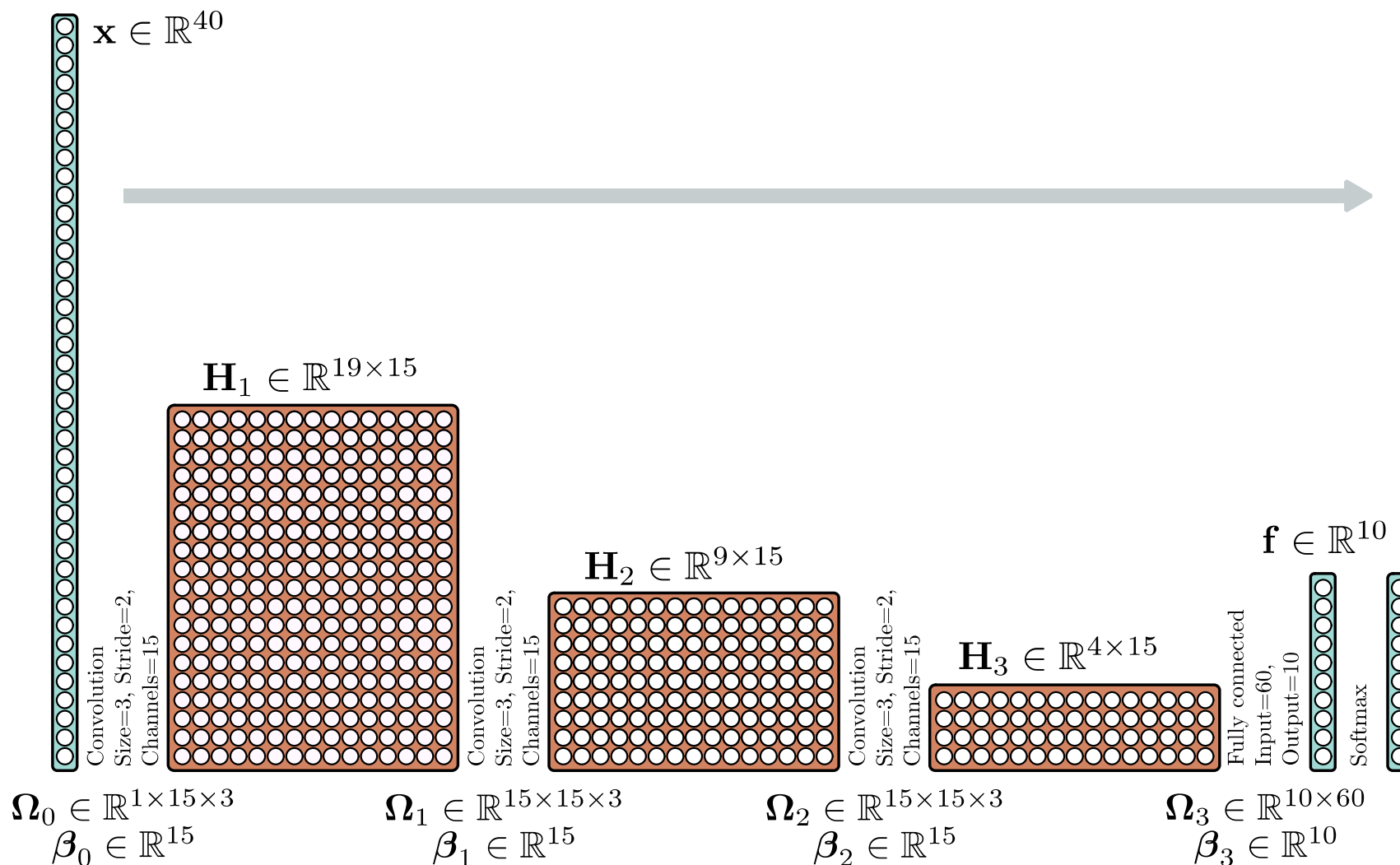


## Convolutional network

- Four hidden layers
- Three convolutional layers
- One fully-connected layer
- Softmax at end
- Total parameters = 2050
- Trained for 100,000 steps with SGD, LR = 0.01, batch size 100



## MNIST-1D convolutional network

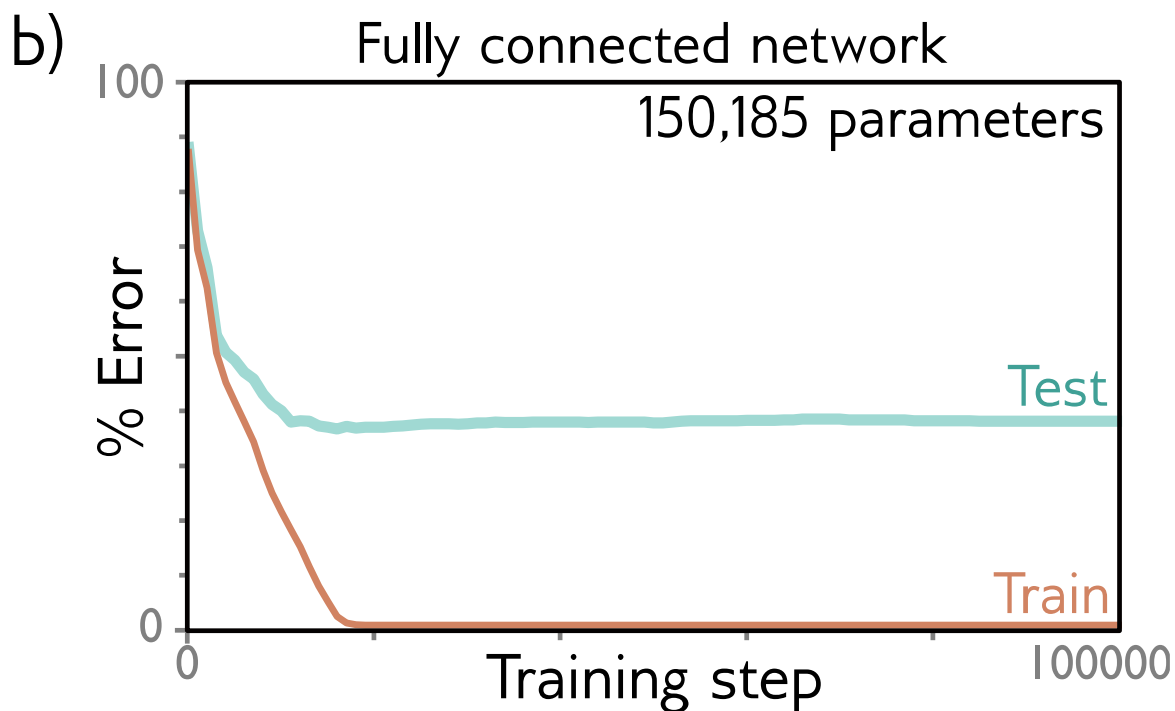
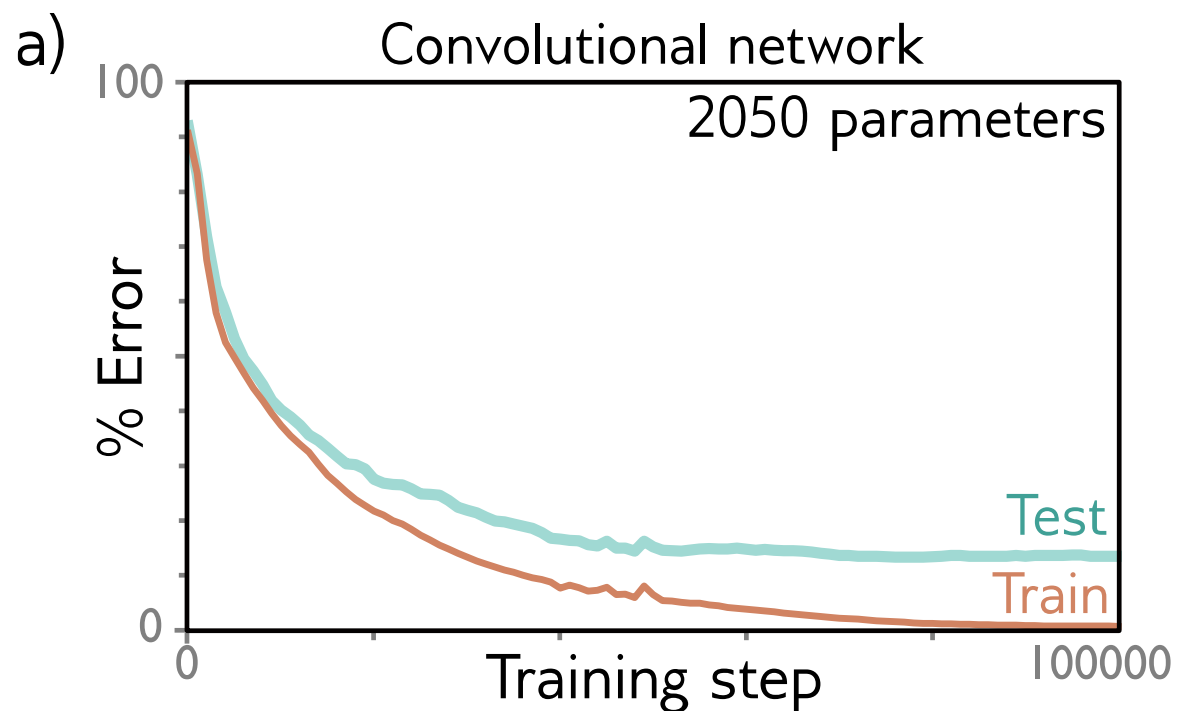




## Fully connected network

- Exactly same number of layers and hidden units
- All fully-connected layers
- Total parameters = 150,185

## Performance





## Why?

- Better **inductive bias**
- Forced the network to process each location similarly
- Shares information across locations
- Search through a smaller family of input/output mappings, all of which are plausible



## Convolution #2

- 2D Convolution
- Downsampling and upsampling, 1x1 convolution
- Image classification
- Object detection
- Semantic segmentation

# 2D Convolution

- The previous section described convolutional networks for processing 1D data.
- Such networks can be applied to financial time series, audio, and text.
- However, convolutional networks are more usually applied to 2D **image data**. The convolutional kernel is now a 2D object.

# 2D Convolution

- Convolution in 2D
  - Weighted sum over a  $K \times K$  region
  - $K \times K$  weights
- Build into a convolutional layer by adding bias and passing through activation function

$$h_{i,j} = a \left[ \beta + \sum_{m=1}^3 \sum_{n=1}^3 \omega_{m,n} x_{i+m-2,j+n-2} \right]$$

where  $\omega_{m,n}$  are the entries of the convolutional kernel.

This is simply a weighted sum over a square  **$3 \times 3$  input region**.

## 2D Convolution

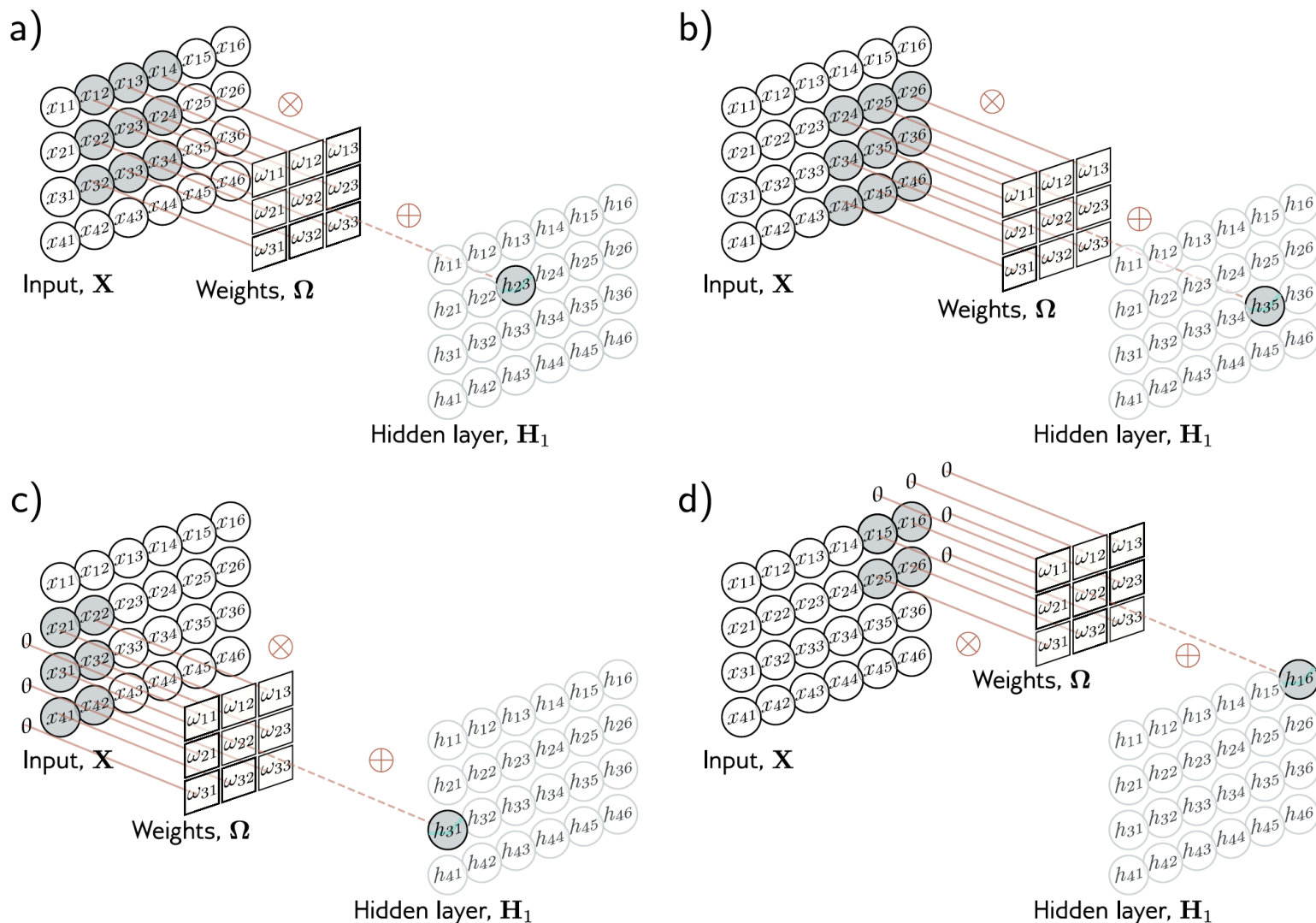


Figure. 2D convolutional layer. Each output  $h_{ij}$  computes a weighted sum of the 3x3 nearest inputs, adds a bias, and passes the result through an activation function.

a) Here, the output  $h_{23}$  (shaded output) is a weighted sum of the nine positions from  $x_{12}$  to  $x_{34}$  (shaded inputs).

b) Different outputs are computed by translating the kernel across the image grid in two dimensions.

c-d) With zero padding, positions beyond the image's edge are considered to be zero.





## Channels in 2D convolution

- Often the input is an RGB image, which is treated as a 2D signal with three channels .
- Here, a  $3 \times 3$  kernel would have  $3 \times 3 \times 3$  weights and be applied to the 2D convolution three input channels at each of the  $3 \times 3$  positions to create a 2D output that is the same height and width as the input image (assuming zero padding).



## Channels in 2D convolution

- To generate multiple output channels, we repeat this process with different kernel weights and append the results to form a **3D tensor**.

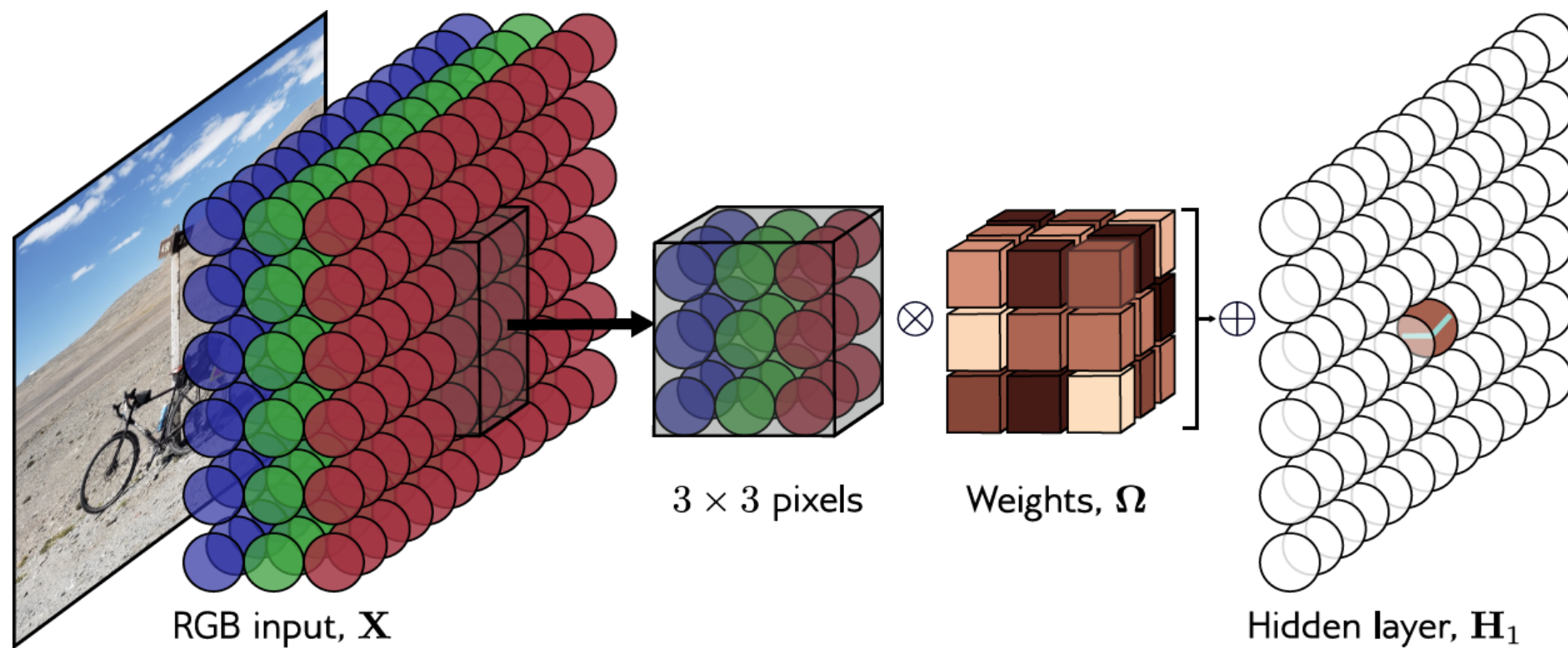
## Channels in 2D convolution

- If the kernel is size  $K \times K$ , and there are  $C_i$  input channels, each output channel is a weighted sum of  $C_i \times K \times K$  quantities plus one bias.

$$\omega \in \mathbb{R}^{C_i \times K \times K} \qquad \beta \in \mathbb{R}$$

- It follows that to compute  $C_o$  output channels, we need  $C_i \times C_o \times K \times K$  weights and  $C_o$  biases.

$$\omega \in \mathbb{R}^{C_i \times C_o \times K \times K} \qquad \beta \in \mathbb{R}^{C_o}$$



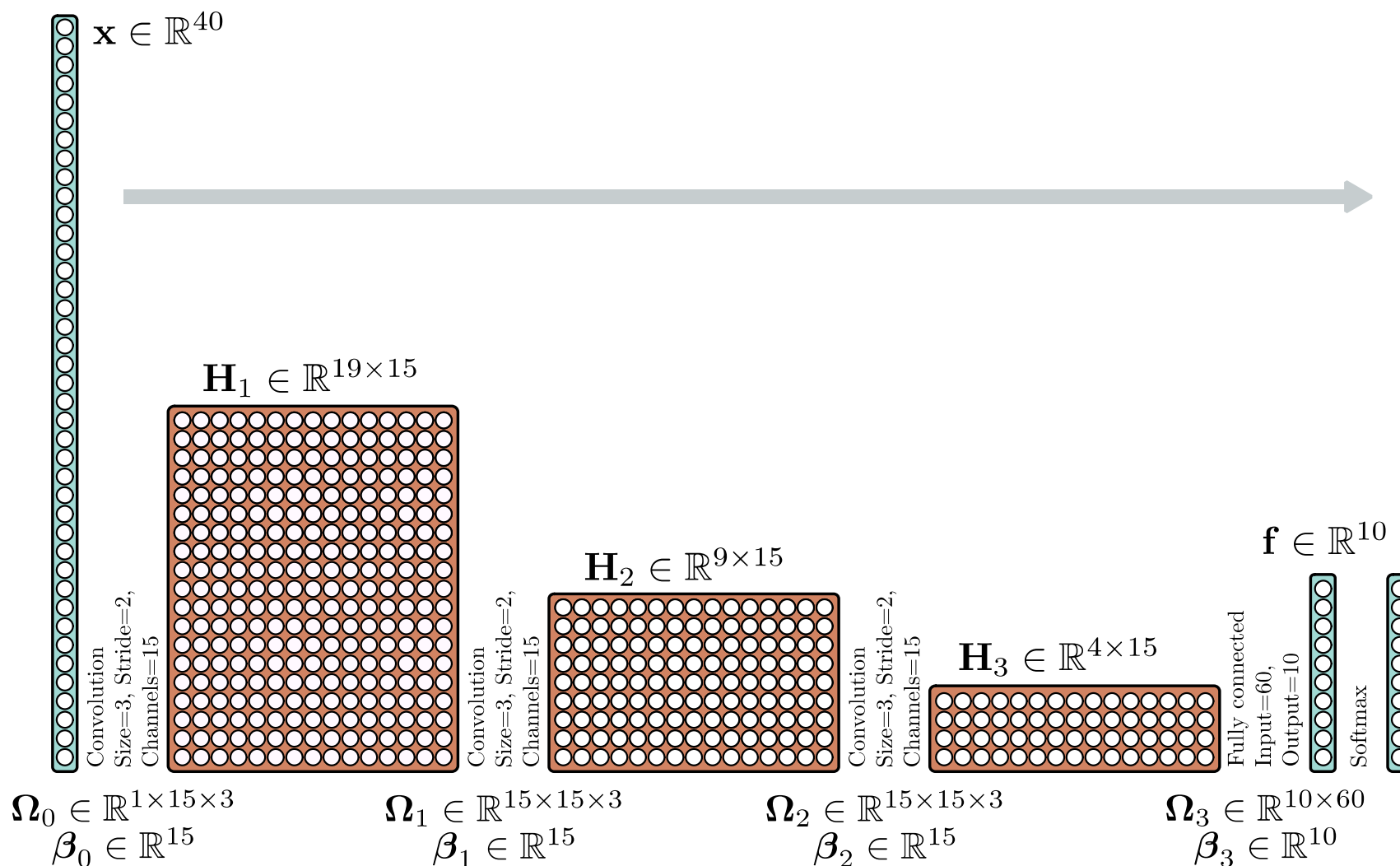
**Figure** 2D convolution applied to an image. The image is treated as a **2D input with three channels** corresponding to the red, green, and blue components. With a  $3 \times 3$  kernel, each pre-activation in the first hidden layer is computed by pointwise multiplying the  $3 \times 3 \times 3$  kernel weights with the  $3 \times 3$  RGB image patch centered at the same position, summing, and adding the bias. To calculate all the pre-activations in the hidden layer, we “slide” the kernel over the image in both horizontal and vertical directions. The output is a 2D layer of hidden units. To create multiple output channels, we would repeat this process with multiple kernels, resulting in a 3D tensor of hidden units at hidden layer  $\mathbf{H}_1$ .



## Convolution #2

- 2D Convolution
- Downsampling and upsampling, 1x1 convolution
- Image classification
- Object detection
- Semantic segmentation

## MNIST-1D convolutional network





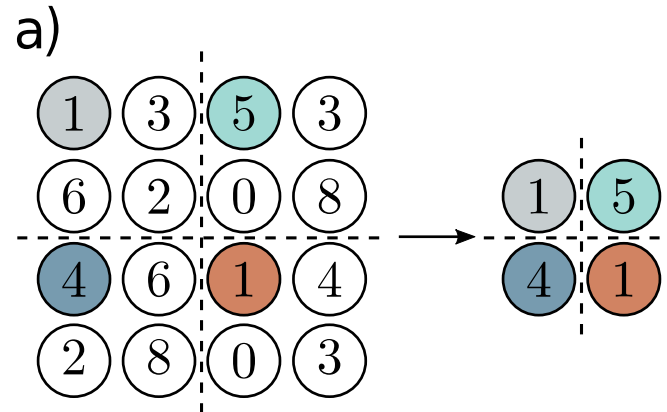
## Downsampling and upsampling

- The network **increased receptive field size** by scaling down the representation at each layer using stride two convolutions.
- We now consider methods for scaling down or **downsampling** 2D input representations.
- We also describe methods for scaling them back up (**upsampling**), which is useful when the output is also an image.
- Finally, we consider methods to change the number of channels between layers. This is helpful when recombining representations from two branches of a network.



## Downsampling

- There are three main approaches to scaling down a 2D representation. Here, we consider the most common case of scaling down both dimensions by a factor of two.
- First, we can **sample every other position**. When we use a stride of two, we effectively apply this method simultaneously with the convolution operation (figure a).

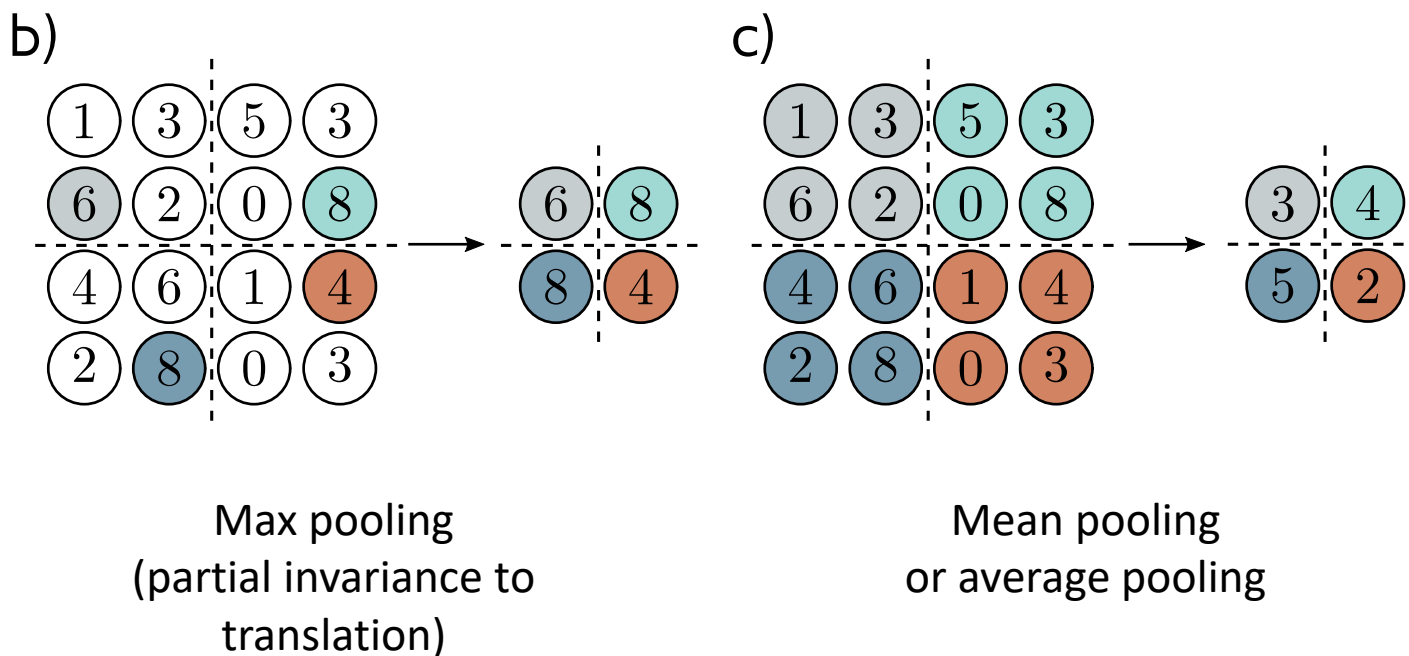


Sample every other position  
(equivalent to stride two)



# Downsampling

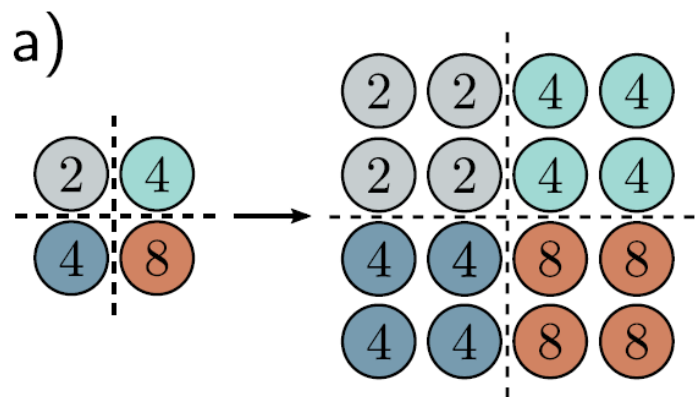
- Second, **max pooling** retains the maximum of the  $2 \times 2$  input values (figure b). This induces some invariance to translation; if the input is shifted by one pixel, many of these maximum values remain the same.
- Finally, **mean pooling** or **average pooling** averages the inputs.
- For all approaches, we apply downsampling separately to each channel, so the output has half the width and height but the same number of channels.



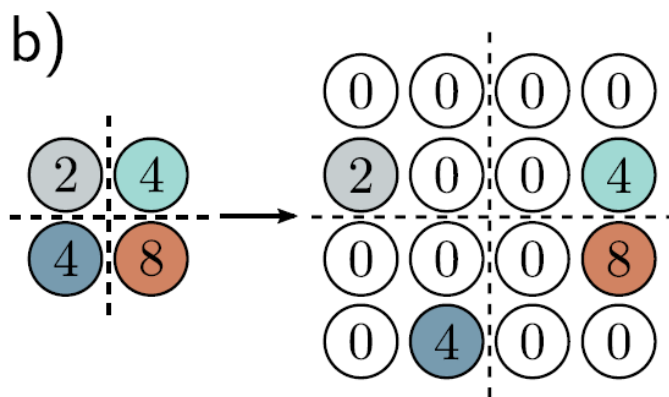


## Upsampling

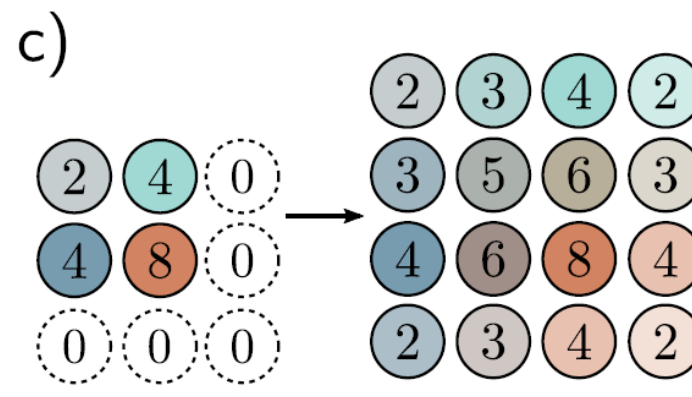
- The simplest way to scale up a network layer to double the resolution is to **duplicate** all the channels at each spatial position four times (figure a).
- A second method is **max unpooling**; this is used where we have previously used a max pooling operation for downsampling, and we distribute the values to the positions they originated from (figure b).
- A third approach uses **bilinear interpolation** to fill in the missing values between the points where we have samples. (figure c).



Duplicate



Max-upsampling

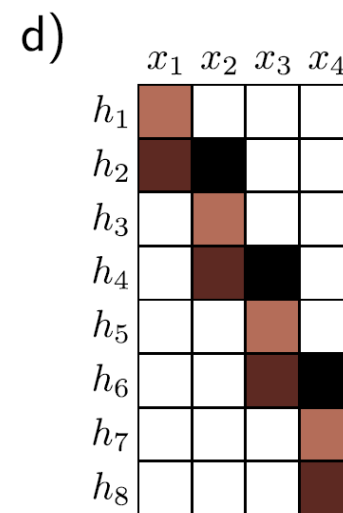
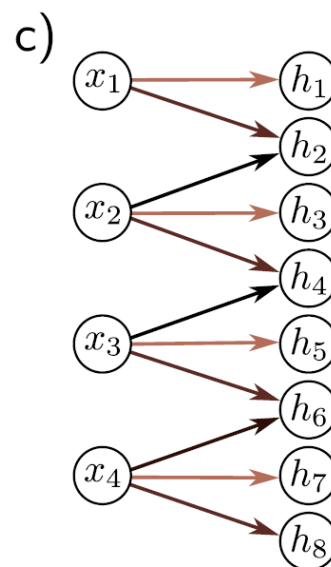
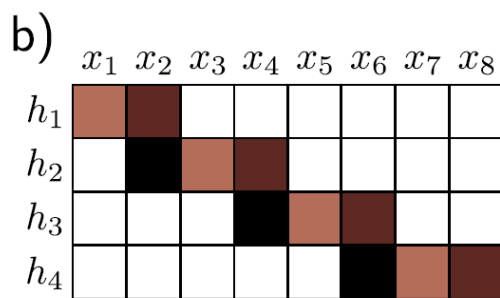
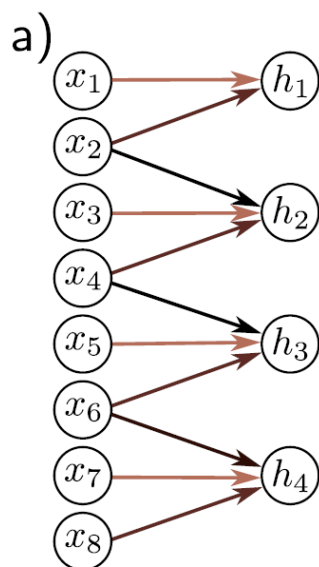


Bilinear interpolation



## Upsampling

- A fourth approach is roughly analogous to downsampling using a stride of two.
  - In that method, there were half as many outputs as inputs, and for kernel size three, each output was a weighted sum of the three closest inputs (figure a).
  - In **transposed convolution**, this picture is reversed (figure c).
  - There are twice as many outputs as inputs, and each input contributes to three of the outputs.
  - When we consider the associated weight matrix of this upsampling mechanism (figure d), we see that it is the transpose of the matrix for the downsampling mechanism (figure b).



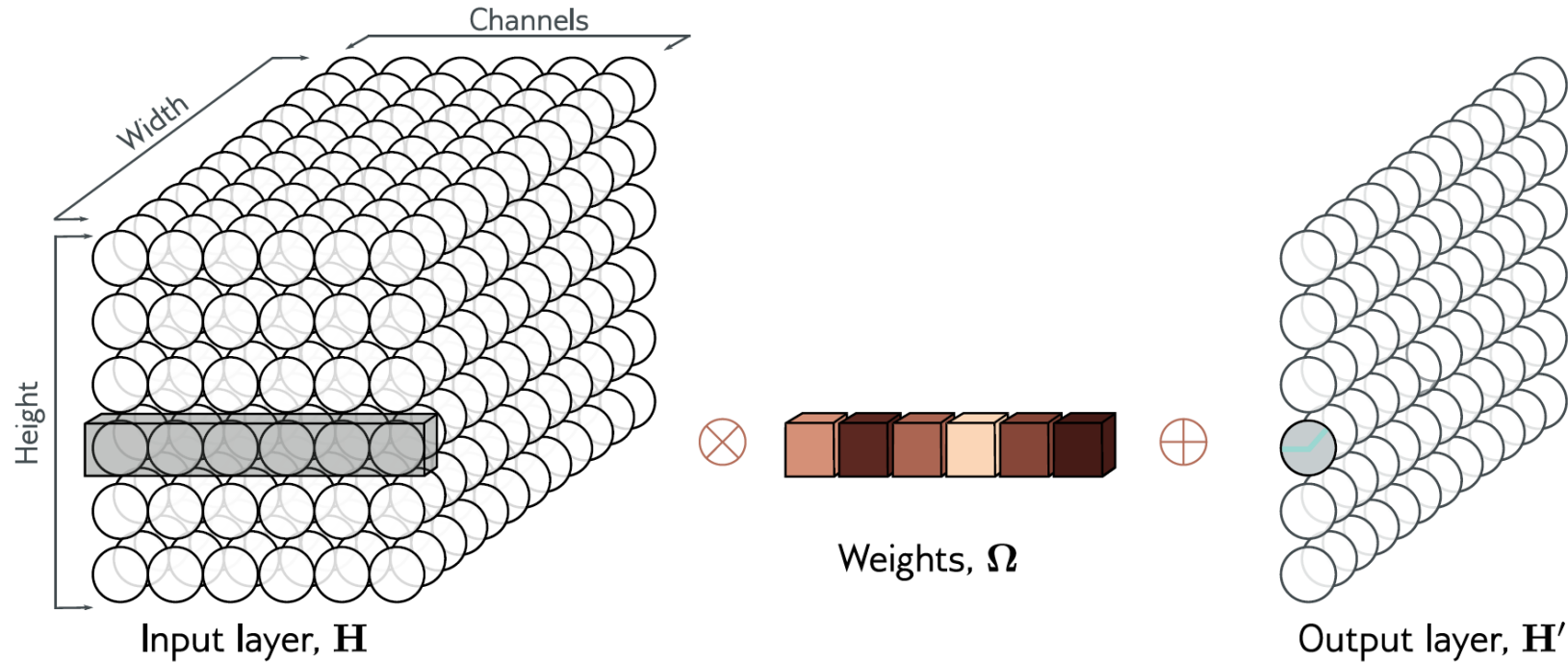


## 1x1 convolution

- Sometimes we want to **change the number of channels** between one hidden layer and the next without further spatial pooling. This is usually so we can combine the representation with another parallel computation.
- To accomplish this, we apply **a convolution with kernel size one**. Each element of the output layer is computed by taking a weighted sum of all the channels at the same position.
- We can repeat this multiple times with different weights to generate as many output channels as we need. The associated convolution weights have size  $1 \times 1 \times C_i \times C_o$ .
- Hence, this is known as **1x1 convolution**. Combined with a bias and activation function, it is equivalent to running the same fully connected network on the channels at every position.



## 1x1 convolution



- Mixes channels
- Can change number of channels
- Equivalent to running same fully connected network at each position



## Convolution #2

- 2D Convolution
- Downsampling and upsampling, 1x1 convolution
- Image classification
- Object detection
- Semantic segmentation



## Image classification

- The goal of image classification is to assign the image to one of a predetermined set of categories.

## ImageNet database

- Much of the pioneering work on deep learning in computer vision focused on image classification using the **ImageNet dataset**.
- This contains
  - 1,281,167 training images,
  - 50,000 validation images,
  - and 100,000 test images,
  - and every image is labeled as belonging to one of 1000 possible categories.



## ImageNet database

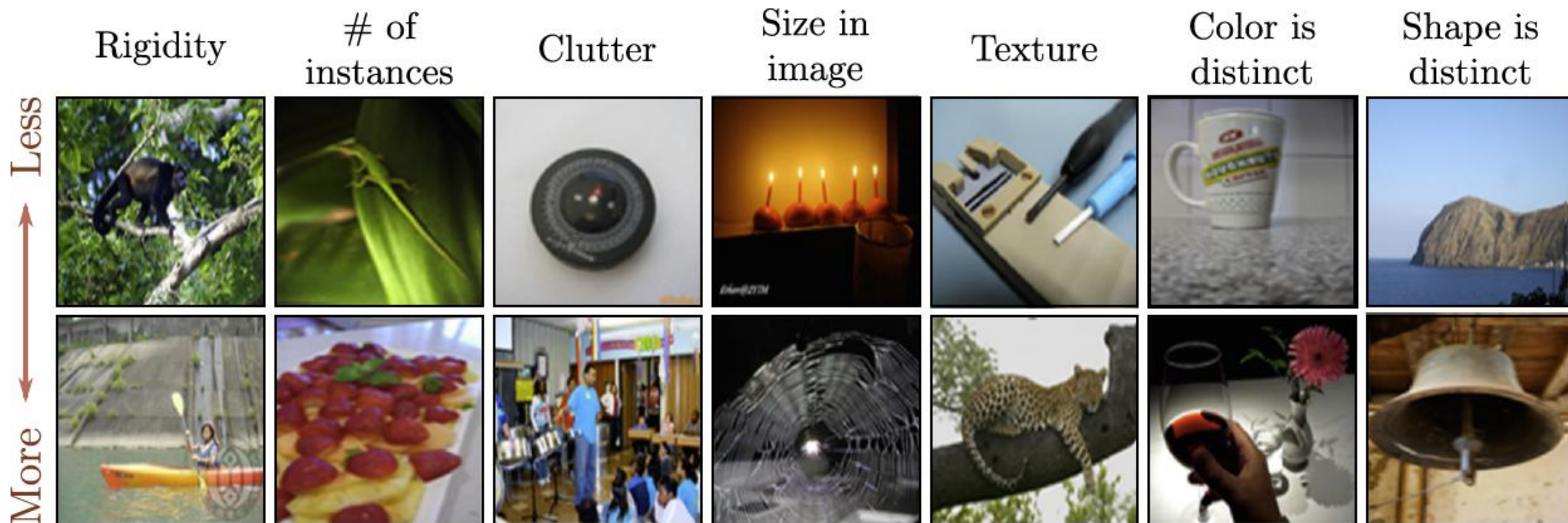
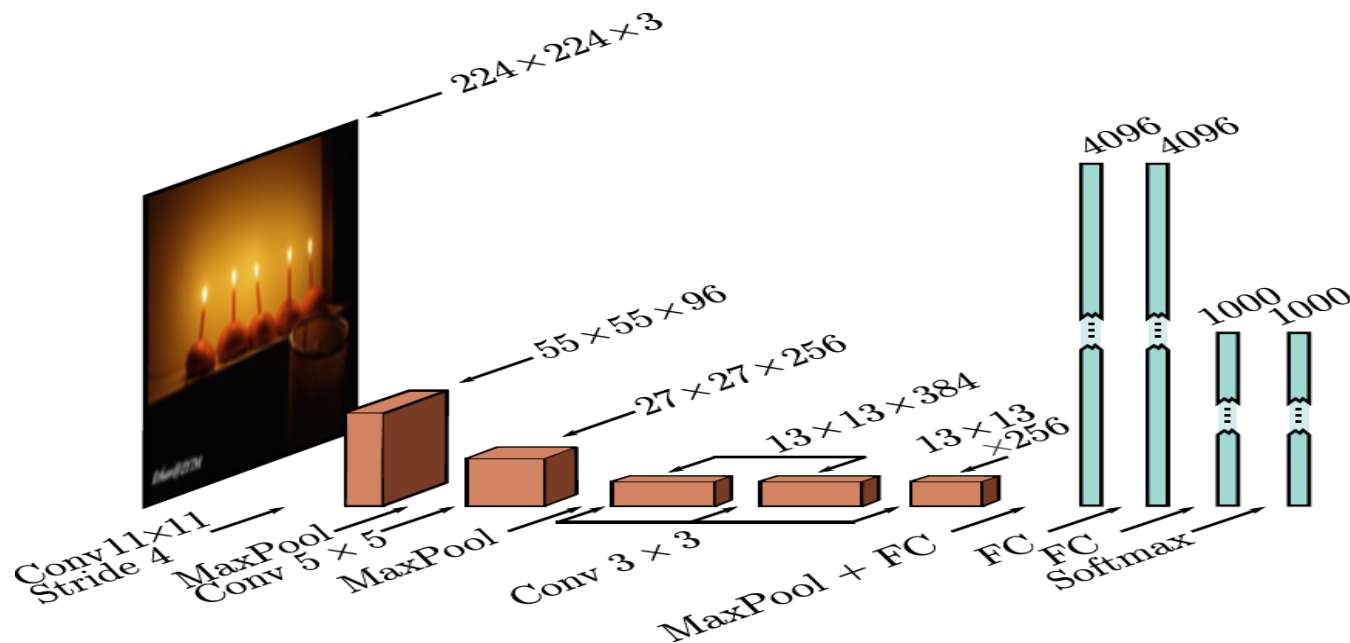


Figure. Example ImageNet classification images. The model aims to assign an input image to one of 1000 classes. This task is challenging because the images vary widely along different attributes (columns). These include rigidity (monkey < canoe), number of instances in image (lizard < strawberry), clutter (compass < steel drum), size (candle < spiderweb), texture (screwdriver < leopard), distinctiveness of color (mug < red wine), and distinctiveness of shape (headland < bell). Adapted from Russakovsky et al. (2015).

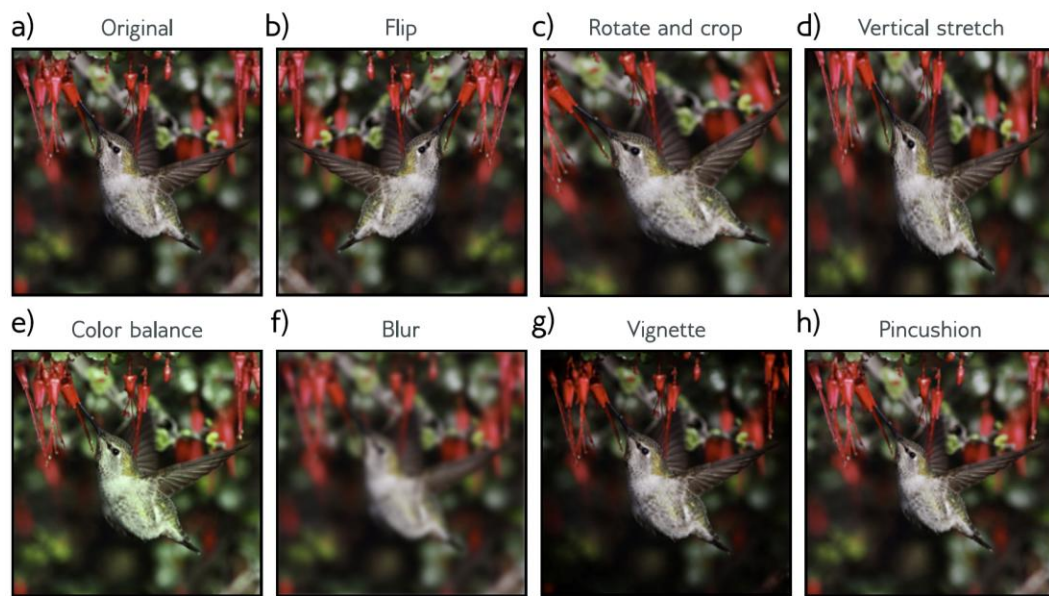
# AlexNet (2012)

- In 2011, before deep networks were applied, the state-of-the-art method classified the test images with  $\sim$  **25% errors** for the correct class being in the **top five suggestions**. Five years later, the best deep learning models eclipsed human performance.
- In 2012, **AlexNet** was the first convolutional network to perform well on this task.
  - It consists of **eight hidden layers** with ReLU activation functions, of which the first five are convolutional and the rest fully connected.

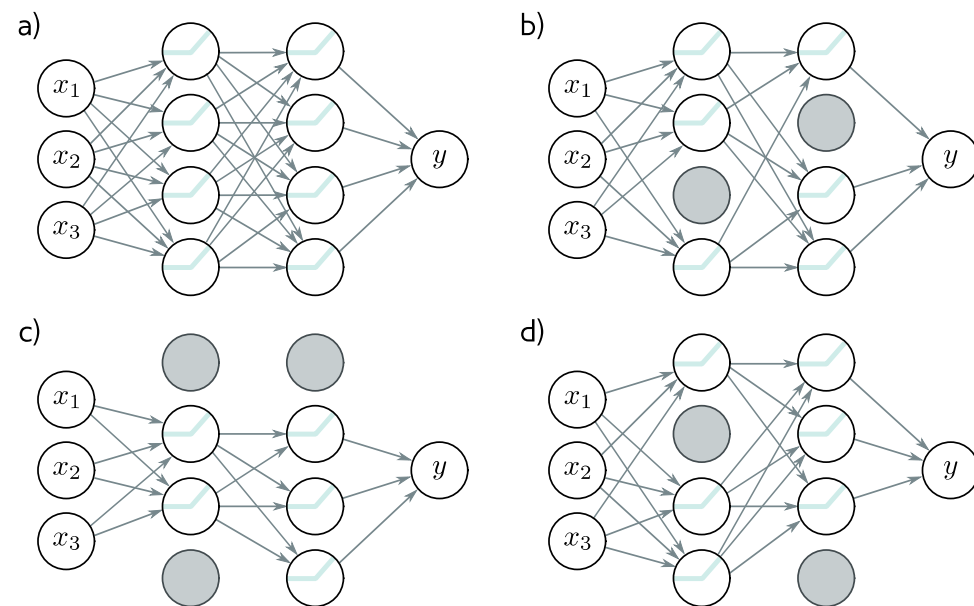


## AlexNet (2012)

- The complete network contains **~60 million parameters**. Most of these are in the fully connected layers and the end of the network.
- At test time average results from five different cropped and mirrored versions of the image SGD with a momentum coefficient of 0.9 and batch size of 128. L2 (weight decay) regularizer used.
- This system achieved a **16.4% top-5 error rate** and a 38.1% top-1 error rate.



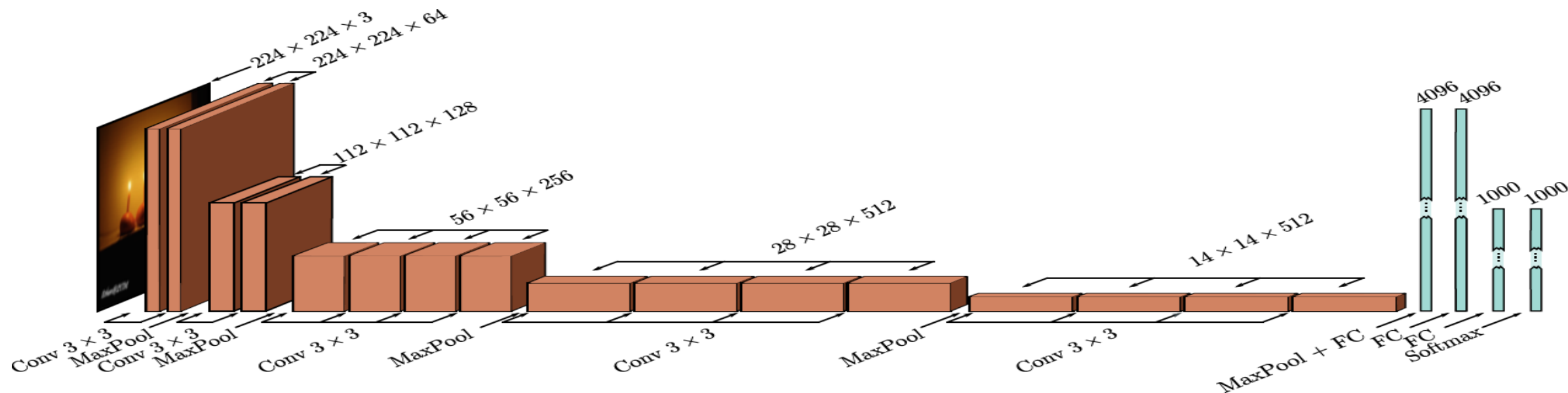
- Data augmentation a factor of 2048 using (i) spatial transformations and (ii) modifications of the input intensities.



- Dropout was applied in the fully connected layers

# VGG (2015)

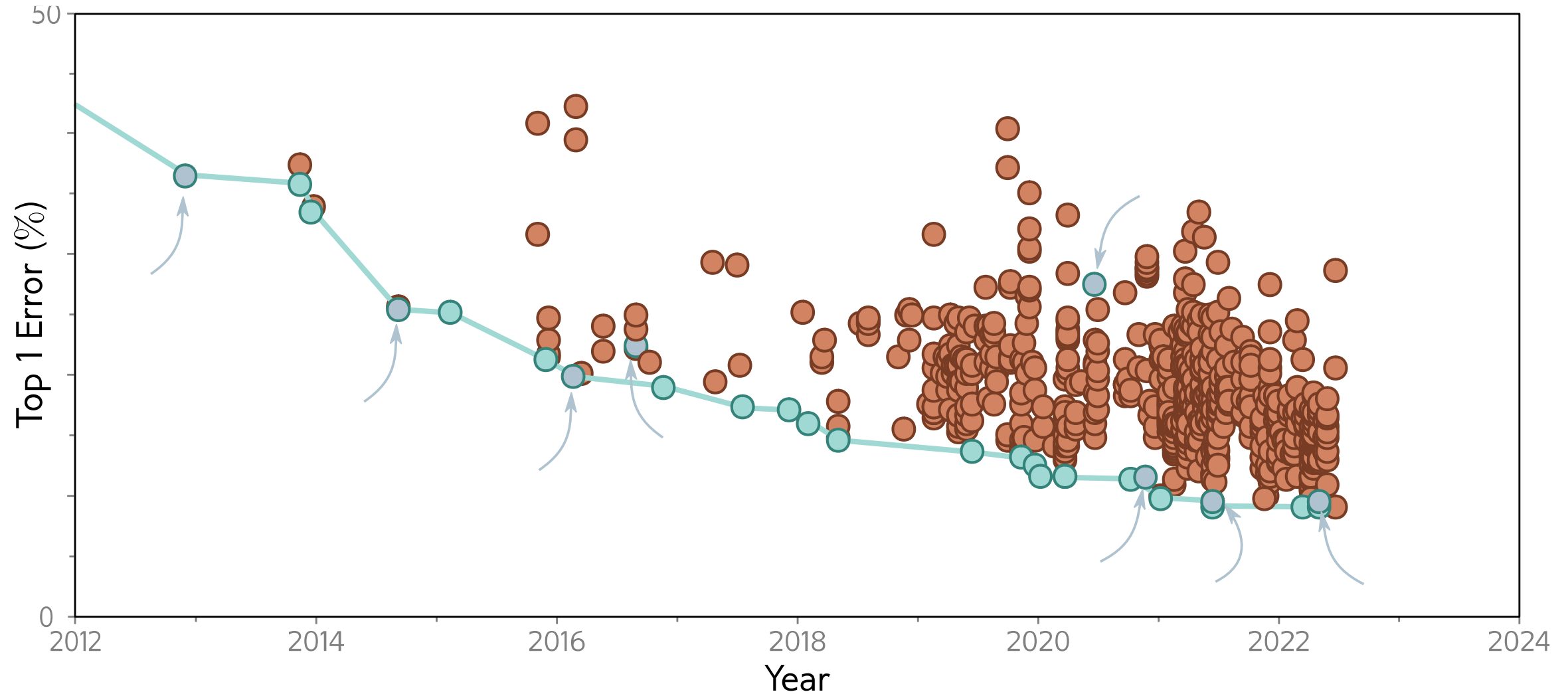
- The VGG network was also targeted at classification in the ImageNet task and achieved a considerably better performance of **6.8% top-5 error rate** and a 23.7% top-1 error rate.
- This network is similarly composed of a series of interspersed convolutional and max pooling layers, where the spatial size of the representation gradually decreases, but the number of channels increase. These are followed by three fully connected layers. The VGG network was also trained using data augmentation, weight decay, and dropout.
- The most important change between AlexNet and VGG was the **depth of the network**. The latter used **19 hidden layers** and **144 million parameters**.







# ImageNet History





## Convolution #2

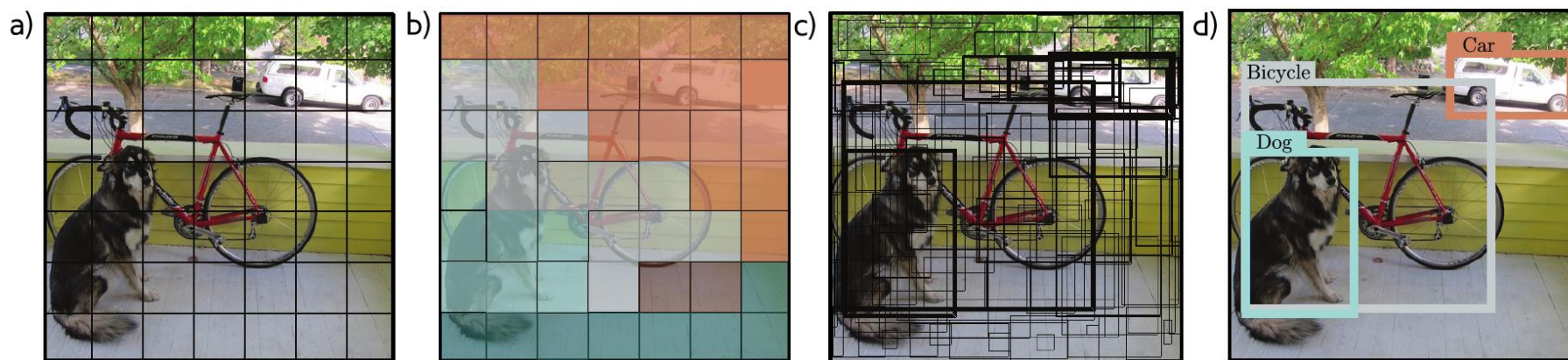
- 2D Convolution
- Downsampling and upsampling, 1x1 convolution
- Image classification
- **Object detection**
- Semantic segmentation

# Object detection

- In object detection, the goal is to **identify and localize multiple objects** within the image.
- An early method based on convolutional networks was **You Only Look Once, or YOLO** for short.
- The input to the YOLO network is a  $448 \times 448$  RGB image.
- This is passed through 24 convolutional layers that gradually decrease the representation size using max pooling operations while concurrently increasing the number of channels, similarly to the VGG network. The final convolutional layer is of size  $7 \times 7$  and has 1024 channels. This is reshaped to a vector, and a fully connected layer maps it to 4096 values. One further fully connected layer maps this representation to the output.

# YOLO

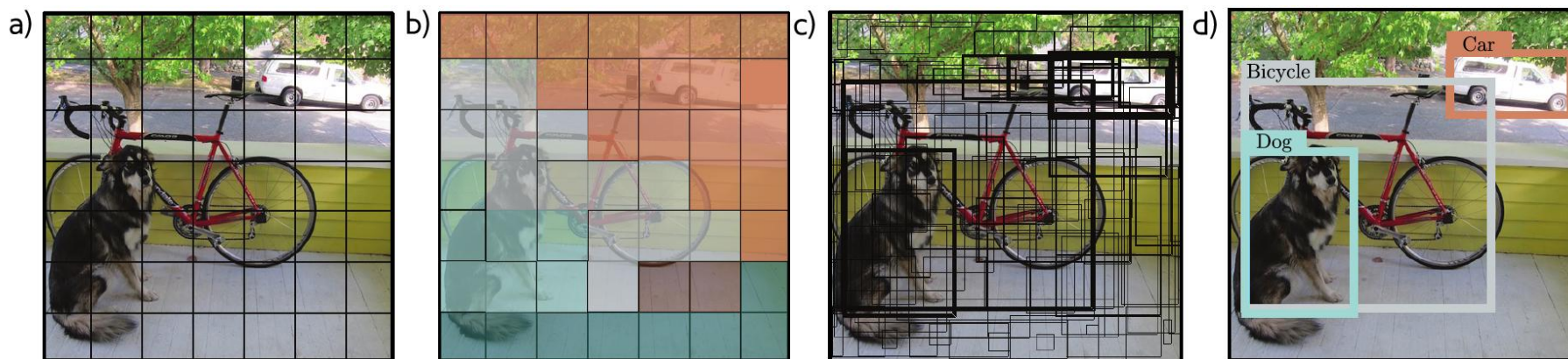
- The output values encode which class is present at each of a  **$7 \times 7$  grid of locations** (figure a–b).
- For each location, the output values also encode a fixed number of **bounding boxes**. Five parameters define each box: the x- and y-positions of the center, the height and width of the box, and the confidence of the prediction (figure c).
- The confidence estimates the overlap between the predicted and ground truth bounding boxes.
- The system is trained using momentum, weight decay, dropout, and data augmentation.
- **Transfer learning** is employed; the network is initially trained on the ImageNet classification task and is then fine-tuned for object detection.





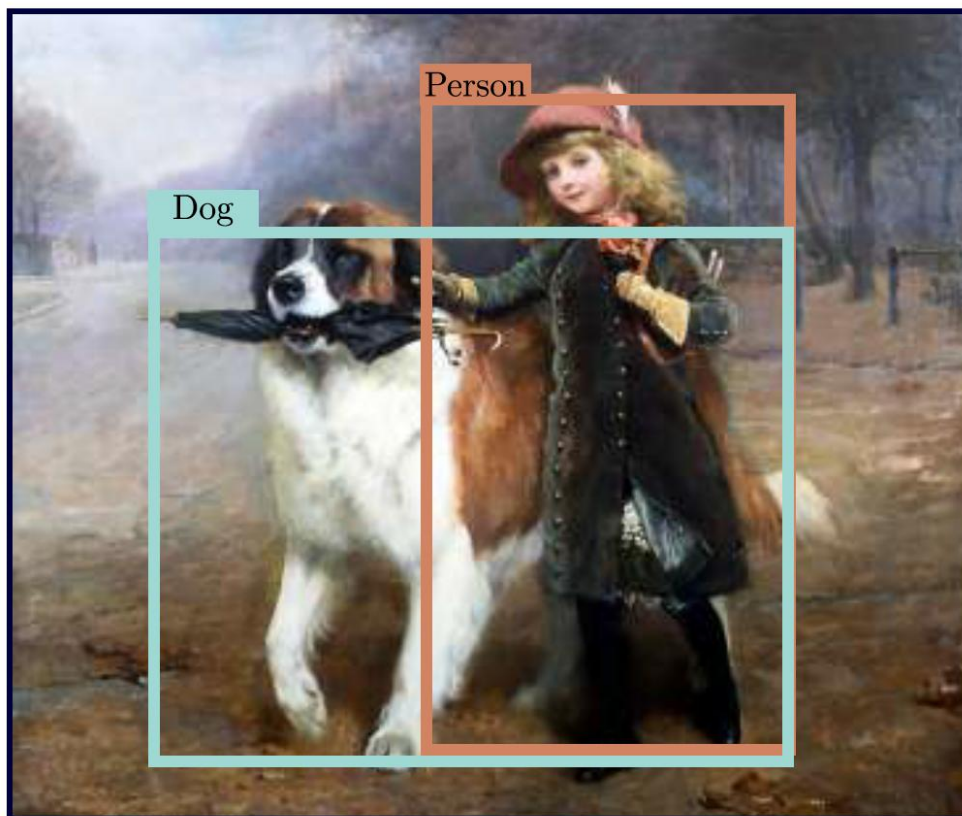
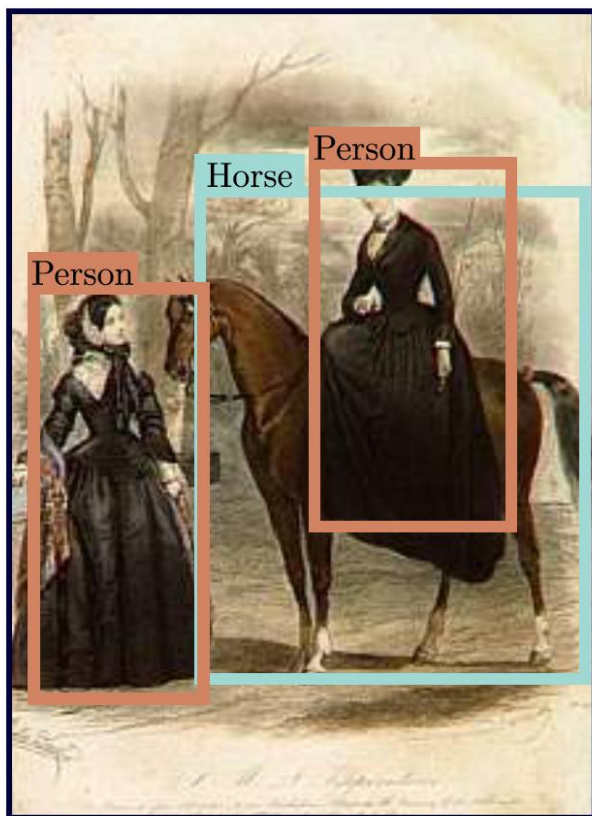
# YOLO

- After the network is run, a **heuristic process** is used to remove rectangles with low confidence and to suppress predicted bounding boxes that correspond to the same object so only the most confident one is retained.

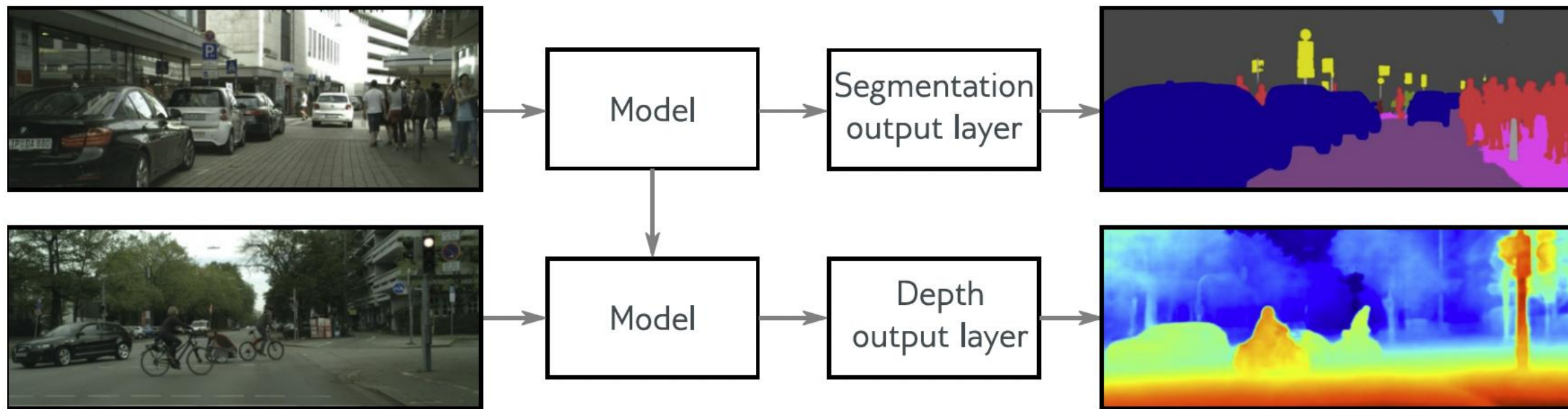




## Results



## Transfer learning

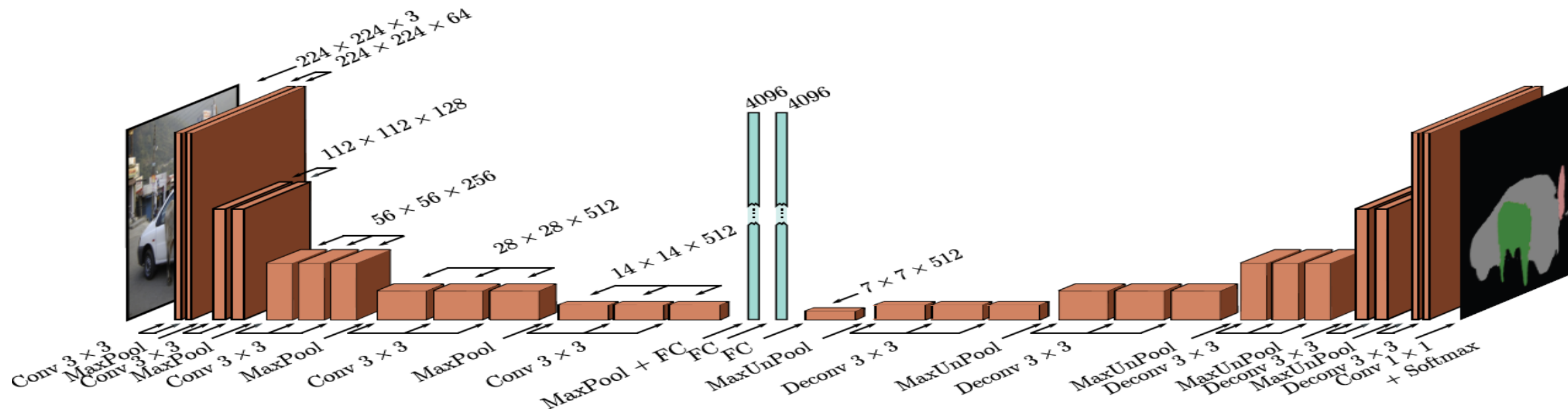


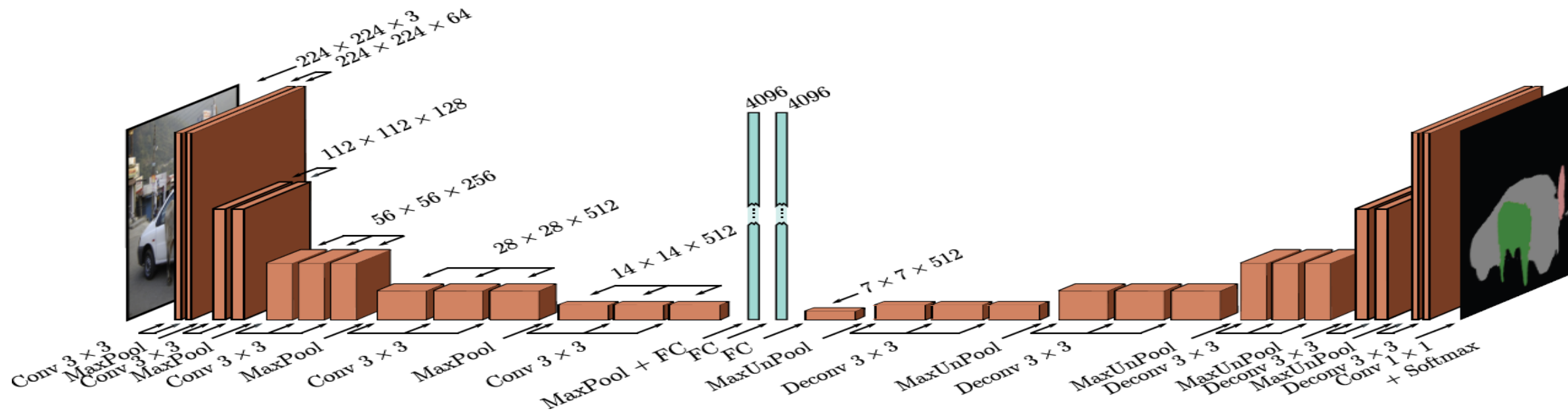
Transfer learning from ImageNet classification



## Convolution #2

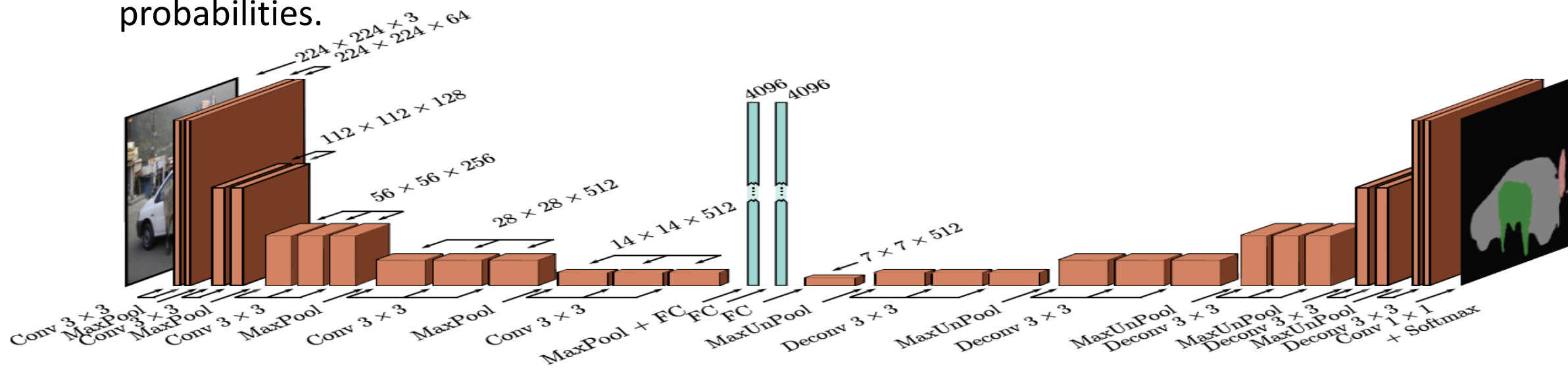
- 2D Convolution
- Downsampling and upsampling, 1x1 convolution
- Image classification
- Object detection
- Semantic segmentation

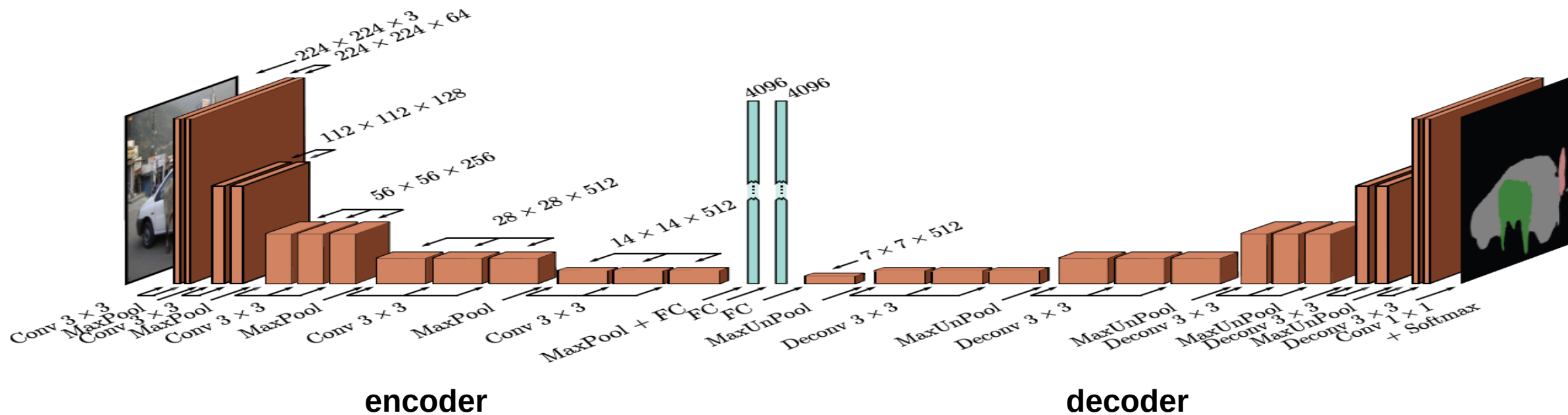




## Semantic Segmentation (2015)

- Here, the architecture **diverges from VGG**.
- Another fully connected layer reconstitutes the representation into  $7 \times 7$  spatial positions and 512 channels.
- This is followed by a series of max unpooling layers and deconvolution layers. These are transposed convolutions but in 2D and without the upsampling.
- Finally, there is a  $1 \times 1$  convolution to create 21 channels representing the possible classes and a softmax operation at each spatial position to map the activations to class probabilities.

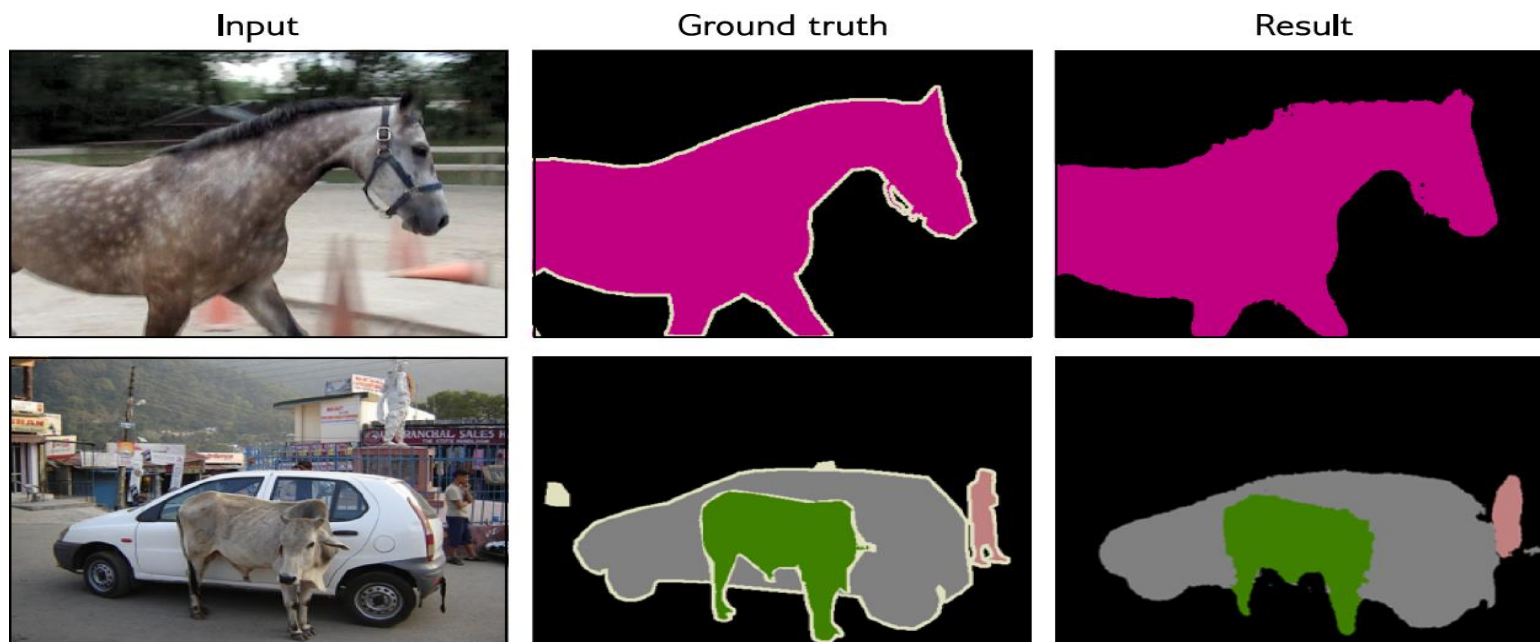






# Semantic Segmentation (2015)

- The final segmentation is generated using a heuristic method that greedily searches for the class that is most represented and infers its region, taking into account the probabilities but also encouraging connectedness.
- Then the next most-represented class is added where it dominates at the remaining unlabeled pixels. This continues until there is insufficient evidence to add more.



# Summary

- In convolutional layers, each hidden unit is computed by taking a weighted sum of the nearby inputs, adding a bias, and applying an activation function.
- The weights and the bias are the same at every spatial position, so there are **far fewer parameters** than in a fully connected network, and the parameters don't increase with the input image size.
- To ensure that information is not lost, this operation is repeated with different weights and biases to create multiple channels at each spatial position.



## Summary

- Typical convolutional networks consist of convolutional layers interspersed with layers that downsample by a factor of two.
- As the network progresses, the spatial dimensions usually decrease by factors of two, and the number of channels increases by factors of two.
- At the end of the network, there are typically one or more fully connected layers that integrate information from across the entire input and create the desired output.
- If the output is an image, a mirrored “decoder” upsamples back to the original size.

# Summary

- The translational equivariance of convolutional layers imposes a useful inductive bias that increases performance for image-based tasks relative to fully connected networks.
- We described image classification, object detection, and semantic segmentation networks. Image classification performance was shown to improve as the network became deeper. However, subsequent experiments showed that increasing the network depth indefinitely doesn't continue to help; after a certain depth, the system becomes difficult to train. This is the motivation for residual connections.